

IF3270 Pembelajaran Mesin

Pembelajaran Mesin Convolutional Neural Network dan Recurrent Neural Network

Laporan Tugas Besar 2

Disusun untuk memenuhi tugas mata kuliah Pembelajaran Mesin pada Semester 6 (enam)

Tahun Akademik 2025/2026



Disusun Oleh:

Ahmad Syafiq	(13523135)
Muhammad Aulia Azka	(13523137)
Fachriza Ahmad Setiyono	(13523162)

**PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
BANDUNG
2026**

DAFTAR ISI

DAFTAR ISI.....	2
BAB I	
DESKRIPSI PERSOALAN.....	4
BAB II	
IMPLEMENTASI.....	5
2.1. Kelas.....	5
2.1.1. Dense.....	5
2.1.2. KerasLocallyConnected2D.....	6
2.1.3. MaxPooling2D.....	9
2.1.4. AveragePooling2D.....	10
2.1.5. Flatten.....	11
2.1.6. Conv2D.....	12
2.1.7. GlobalAveragePooling2D.....	15
2.1.8. GlobalMaxPooling2D.....	15
2.1.9. LocallyConnected2D.....	16
2.1.10. ScratchSequential.....	19
2.1.11. Embedding.....	20
2.1.12. SimpleRNNCell.....	21
2.1.13. SimpleRNN.....	22
2.1.14. LSTMCell.....	23
2.1.15. LSTM.....	24
2.1.16. ScratchDecoderInference.....	26
2.2. Fungsi Pendukung.....	30
2.2.1. Fungsi Aktivasi.....	30
2.2.2. Utility Pemrosesan Gambar.....	31
2.2.3. Konfigurasi Eksperimen CNN.....	32
2.2.4. Builder Model CNN Keras.....	32
2.2.5. Training dan Evaluasi CNN.....	32
2.2.6. Loader Bobot CNN.....	33
2.2.7. Preprocessing Caption.....	33
2.2.8. Feature Extraction RNN/LSTM.....	34
2.2.9. Decoder Keras RNN/LSTM.....	34
2.2.10. Evaluasi dan Inference Caption.....	34
2.3. Alur Implementasi.....	35
2.3.1. Alur CNN.....	35
2.3.2. Alur RNN/LSTM.....	35
BAB III	
PENJELASAN FORWARD PROPAGATION.....	36
3.1. CNN.....	36
3.1.1. Persiapan Input.....	37
3.1.2. Forward Propagation Conv2D Shared Parameter.....	37
3.1.3. Padding pada Conv2D.....	38
3.1.4. Forward Propagation LocallyConnected2D Non-Shared Parameter.....	38

3.1.5. Forward Propagation Pooling.....	39
3.1.6. Forward Propagation Flatten dan Dense.....	39
3.2. RNN.....	40
3.2.1. Input dan Representasi Data.....	41
3.2.2. Proyeksi Feature CNN.....	41
3.2.3. Embedding Token Caption.....	41
3.2.4. Forward Propagation.....	42
3.2.7. Dense Output dan Softmax.....	42
3.3. LSTM.....	42
3.3.1. State pada LSTM.....	43
3.3.3. Input Gate.....	43
3.3.4. Forget Gate.....	43
3.3.5. Candidate Cell.....	44
3.3.6. Pembaruan Cell State.....	44
3.3.7. Output Gate dan Hidden State.....	44
3.3.9. Multi-layer LSTM.....	44
BAB IV	
HASIL PENGUJIAN.....	45
4.1. CNN.....	45
4.1.1. Ringkasan Hasil 16 Eksperimen Conv2D.....	45
4.1.2. Analisis Model Terbaik.....	48
4.1.3. Pengaruh Jumlah Layer Konvolusi.....	49
4.2. RNN dan LSTM Untuk Image Captioning.....	49
4.2.1. Perbandingan jumlah layer: RNN & LSTM.....	49
4.2.2. Perbandingan ukuran hidden state: RNN & LSTM.....	49
4.2.3. Perbandingan RNN vs LSTM.....	50
4.2.4. Perbandingan Keras vs from scratch.....	51
4.2.5. Pengaruh panjang maksimum caption terhadap BLEU-4.....	51
BAB V	
KESIMPULAN & SARAN.....	53
Kesimpulan.....	53
Saran.....	53
REFERENSI.....	54
LAMPIRAN.....	55
Link Repositori.....	55
Tabel Pembagian Tugas.....	55
Form Penggunaan AI.....	56

BAB I

DESKRIPSI PERSOALAN

Tugas Besar II pada kuliah IF3270 Pembelajaran Mesin bertujuan agar peserta kuliah mendapatkan wawasan tentang bagaimana cara mengimplementasikan [Convolutional Neural Network \(CNN\)](#) dan [Recurrent Neural Network \(RNN\)](#). Pada tugas ini, peserta kuliah akan ditugaskan untuk mengimplementasikan modul *forward propagation* CNN, RNN, dan LSTM *from scratch*. Selain itu, peserta juga akan mengeksplorasi pipeline *image captioning* melalui arsitektur encoder-decoder yang menggabungkan CNN dan LSTM, yang merupakan fondasi dari banyak sistem deep learning modern seperti LLM multimodal dan speech recognition.

BAB II IMPLEMENTASI

Implementasi inti program Convolutional Neural Network (CNN), Recurrent Neural Network (RNN), dan Long Short-Term Memory (LSTM) menggunakan Python dengan paradigma berorientasi objek dan prosedural. Terdapat 16 kelas yang tersebar pada modul `common`, `cnn`, dan `rnn_lstm`.

Pustaka yang digunakan adalah NumPy, TensorFlow/Keras, Pillow, pandas, matplotlib, seaborn, scikit-learn, NLTK, tqdm, Jupyter, dan ipykernel. NumPy digunakan sebagai pustaka matematika utama dalam implementasi forward propagation from scratch. TensorFlow/Keras digunakan untuk training model pembandingan, penyimpanan model, dan pemuatan bobot hasil training. scikit-learn dan NLTK digunakan untuk evaluasi, sedangkan Pillow digunakan untuk pemrosesan gambar.

Untuk kejelasan, bab ini membahas deskripsi kelas pada tingkat tinggi beserta atribut dan metodenya. Fungsi-fungsi pendukung yang membentuk pipeline training, evaluasi, preprocessing, dan visualisasi dijelaskan setelah bagian kelas.

2.1. Kelas

2.1.1. Dense

Dense adalah kelas fully connected layer sederhana yang digunakan pada implementasi from scratch. Kelas ini dipakai oleh scratch CNN untuk layer akhir klasifikasi dan oleh scratch decoder RNN/LSTM untuk proyeksi feature serta output vocabulary.

2.1.1.1. Atribut

weights

Menyimpan matriks bobot layer dengan shape (input_dim, output_dim).

bias

Menyimpan vektor bias dengan shape (output_dim,).

activation

Menyimpan nama fungsi aktivasi yang akan diterapkan pada output layer. Nilainya dapat berupa None, relu, softmax, sigmoid, atau tanh.

2.1.1.2. Metode

__init__(self, weights, bias, activation=None)

Menginisialisasi layer dengan bobot, bias, dan aktivasi.

```
def __init__(self, weights, bias, activation=None):
```

```
self.weights = weights
self.bias = bias
self.activation = activation
```

forward(self, x)

Menghitung operasi linear, kemudian menerapkan aktivasi jika diberikan.

```
def forward(self, x):
    out = np.dot(x, self.weights) + self.bias
    return apply_activation(out, self.activation)
```

2.1.2. KerasLocallyConnected2D

KerasLocallyConnected2D adalah custom layer Keras untuk melakukan operasi konvolusi tanpa parameter sharing. Layer ini digunakan untuk membangun model non-shared parameter yang menjadi pembanding terhadap Conv2D biasa.

2.1.2.1. Atribut

filters

Jumlah channel output yang dihasilkan layer.

kernel_size

Ukuran kernel lokal dalam format (height, width).

strides

Ukuran pergeseran kernel pada arah tinggi dan lebar.

padding

Jenis padding yang digunakan, yaitu valid atau same.

activation_name

Nama aktivasi yang digunakan oleh layer.

activation

Objek fungsi aktivasi Keras yang dibangun dari activation_name.

output_h

Tinggi feature map output yang dihitung saat layer dibangun.

output_w

Lebar feature map output yang dihitung saat layer dibangun.

num_positions

Jumlah posisi output, yaitu $\text{output_h} * \text{output_w}$.

patch_size

Banyak elemen pada setiap patch input, yaitu $\text{kernel_height} * \text{kernel_width} * \text{input_channel}$.

kernel

Bobot trainable dengan shape (num_positions, patch_size, filters). Berbeda dengan Conv2D, setiap posisi output memiliki bobotnya sendiri.

bias

Bias trainable dengan shape (num_positions, filters).

2.1.2.2. Metode

`__init__(self, filters, kernel_size, strides=(1, 1), padding="valid", activation=None, **kwargs)`

Menginisialisasi konfigurasi layer non-shared parameter.

```
def __init__(
    self,
    filters,
    kernel_size,
    strides=(1, 1),
    padding="valid",
    activation=None,
    **kwargs
):
    super().__init__(**kwargs)

    self.filters = filters
    self.kernel_size = tuple(kernel_size)
    self.strides = tuple(strides)
    self.padding = padding.lower()
    self.activation_name = activation
    self.activation = tf.keras.activations.get(activation)
```

build(self, input_shape)

Menghitung ukuran output, jumlah posisi output, ukuran patch, serta membuat bobot kernel dan bias.

call(self, inputs)

Melakukan forward pass dengan mengekstraksi patch menggunakan `tf.image.extract_patches`, menghitung output menggunakan `tf.einsum`, menambah bias, melakukan reshape ke bentuk feature map, dan menerapkan aktivasi.

```
def build(self, input_shape):
    _, input_h, input_w, input_channels = input_shape

    if input_h is None or input_w is None or input_channels is None:
        raise ValueError("Input height, width, dan channel harus diketahui.")

    kh, kw = self.kernel_size
    sh, sw = self.strides

    if self.padding == "valid":
        output_h = (input_h - kh) // sh + 1
        output_w = (input_w - kw) // sw + 1
    elif self.padding == "same":
        output_h = math.ceil(input_h / sh)
        output_w = math.ceil(input_w / sw)
    else:
        raise ValueError(f"Padding tidak dikenal: {self.padding}")

    self.output_h = output_h
    self.output_w = output_w
    self.num_positions = output_h * output_w
    self.patch_size = kh * kw * input_channels

    self.kernel = self.add_weight(
        name="kernel",
        shape=(self.num_positions, self.patch_size, self.filters),
        initializer="glorot_uniform",
        trainable=True,
    )
```



```

        self.bias = self.add_weight(
            name="bias",
            shape=(self.num_positions, self.filters),
            initializer="zeros",
            trainable=True,
        )

    super().build(input_shape)

```

get_config(self)

Mengembalikan konfigurasi layer agar model dapat disimpan dan dimuat ulang.

```

def get_config(self):
    config = super().get_config()
    config.update({
        "filters": self.filters,
        "kernel_size": self.kernel_size,
        "strides": self.strides,
        "padding": self.padding,
        "activation": self.activation_name,
    })
    return config

```

2.1.3. MaxPooling2D

MaxPooling2D adalah layer pooling from scratch yang mengambil nilai maksimum pada setiap window spasial. Layer ini mendukung input satu gambar maupun batch.

2.1.3.1. Atribut

pool_size

Ukuran window pooling.

strides

Besar langkah perpindahan window. Jika tidak diberikan, nilainya sama dengan pool_size.

2.1.3.2. Metode

__init__(self, pool_size=(2, 2), strides=None)

Menginisialisasi ukuran window dan stride pooling.

```
def __init__(self, pool_size=(2, 2), strides=None):
    self.pool_size = pool_size
    self.strides = strides if strides is not None else pool_size
```

forward(self, x)

Membuat sliding window dari input menggunakan `sliding_window_view`, lalu mengambil nilai maksimum pada setiap window.

```
def forward(self, x):
    is_single = False

    if x.ndim == 3:
        x = np.expand_dims(x, axis=0)
        is_single = True

    ph, pw = self.pool_size
    sh, sw = self.strides

    windows = sliding_window_view(x, window_shape=(ph, pw),
axis=(1, 2))
    windows = windows[:, ::sh, ::sw, :, :, :]

    output = np.max(windows, axis=(-2, -1))

    if is_single:
        return output[0]

    return output
```

2.1.4. AveragePooling2D

AveragePooling2D adalah layer pooling from scratch yang menghitung rata-rata pada setiap window spasial.

2.1.4.1. Atribut

pool_size

Ukuran window pooling.

strides

Besar langkah perpindahan window.

2.1.4.2. Metode

`__init__(self, pool_size=(2, 2), strides=None)`

Menginisialisasi ukuran window dan stride pooling.

```
def __init__(self, pool_size=(2, 2), strides=None):
    self.pool_size = pool_size
    self.strides = strides if strides is not None else pool_size
```

`forward(self, x)`

Membuat sliding window dari input, lalu menghitung rata-rata nilai pada setiap window.

```
def forward(self, x):
    is_single = False

    if x.ndim == 3:
        x = np.expand_dims(x, axis=0)
        is_single = True

    ph, pw = self.pool_size
    sh, sw = self.strides

    windows = sliding_window_view(x, window_shape=(ph, pw),
axis=(1, 2))
    windows = windows[:, ::sh, ::sw, :, :, :]

    output = np.mean(windows, axis=(-2, -1))

    if is_single:
        return output[0]

    return output
```

2.1.5. Flatten

Flatten adalah layer transisi yang mengubah feature map multidimensi menjadi vektor. Layer ini digunakan sebelum Dense layer pada arsitektur CNN.

2.1.5.1. Atribut

Kelas ini tidak menyimpan atribut trainable maupun konfigurasi khusus.

2.1.5.2. Metode

forward(self, x)

Jika input berupa satu gambar dengan shape (H, W, C), output diubah menjadi vektor 1D. Jika input berupa batch dengan shape (N, H, W, C), output diubah menjadi shape (N, H * W * C).

```
def forward(self, x):
    if x.ndim == 3:
        return x.reshape(-1)

    if x.ndim == 4:
        batch_size = x.shape[0]
        return x.reshape(batch_size, -1)

    raise ValueError(f"Flatten hanya menerima input 3D atau 4D,
dapat shape {x.shape}")
```

2.1.6. Conv2D

Conv2D adalah implementasi layer konvolusi shared parameter from scratch. Bobot yang digunakan berasal dari model Keras sehingga hasil forward propagation dapat dibandingkan langsung dengan hasil model Keras.

2.1.6.1. Atribut

weights

Kernel konvolusi dengan shape (kernel_height, kernel_width, input_channel, output_channel).

bias

Bias untuk setiap output channel.

strides

Langkah pergeseran kernel pada arah tinggi dan lebar.

padding

Jenis padding yang digunakan. Implementasi mendukung valid dan same.

activation

Nama aktivasi yang diterapkan setelah operasi konvolusi.

2.1.6.2. Metode

```
# __init__(self, weights, bias, strides=(1, 1), padding="valid", activation=None)
```

Menginisialisasi bobot, bias, stride, padding, dan aktivasi.

```
def __init__(self, weights, bias, strides=(1, 1), padding="valid",
activation=None):
    self.weights = weights
    self.bias = bias
    self.strides = strides
    self.padding = padding
    self.activation = activation
```

_pad_input(self, x)

Memberikan padding pada input sesuai mode valid atau same.

```
def _pad_input(self, x):
    if self.padding == "valid":
        return x

    if self.padding == "same":
        _, h, w, _ = x.shape
        kh, kw, _, _ = self.weights.shape
        sh, sw = self.strides

        out_h = int(np.ceil(h / sh))
        out_w = int(np.ceil(w / sw))

        pad_h = max((out_h - 1) * sh + kh - h, 0)
        pad_w = max((out_w - 1) * sw + kw - w, 0)

        pad_top = pad_h // 2
        pad_bottom = pad_h - pad_top
        pad_left = pad_w // 2
        pad_right = pad_w - pad_left

        return np.pad(
            x,
            (
                (0, 0),
                (pad_top, pad_bottom),
                (pad_left, pad_right),
                (0, 0),
            ),
```

```

        mode="constant",
        constant_values=0,
    )

    raise ValueError(f"Padding tidak dikenal: {self.padding}")

```

forward(self, x)

Menerima input 3D atau 4D, membentuk sliding window, menghitung konvolusi dengan np.tensordot, menambahkan bias, lalu menerapkan aktivasi.

```

def forward(self, x):
    is_single = False

    if x.ndim == 3:
        x = np.expand_dims(x, axis=0)
        is_single = True

    x_padded = self._pad_input(x)

    kh, kw, _, _ = self.weights.shape
    sh, sw = self.strides

    windows = sliding_window_view(
        x_padded,
        window_shape=(kh, kw),
        axis=(1, 2)
    )

    windows = windows[:, ::sh, ::sw, :, :, :]

    # Dari shape:
    # (N, out_h, out_w, C_in, kh, kw)
    # menjadi:
    # (N, out_h, out_w, kh, kw, C_in)
    windows = np.moveaxis(windows, 3, -1)

    output = np.tensordot(
        windows,
        self.weights,
        axes=([3, 4, 5], [0, 1, 2])
    )

```

```

    )

    output = output + self.bias
    output = apply_activation(output, self.activation)

    if is_single:
        return output[0]

    return output

```

2.1.7. GlobalAveragePooling2D

GlobalAveragePooling2D adalah layer pooling global from scratch yang mereduksi seluruh dimensi spasial dengan rata-rata.

2.1.7.1. Atribut

Kelas ini tidak memiliki atribut trainable.

2.1.7.2. Metode

forward(self, x)

Untuk input 3D, method ini menghitung rata-rata pada axis tinggi dan lebar. Untuk input 4D, rata-rata dihitung pada axis spasial setiap instance batch.

```

def forward(self, x):
    if x.ndim == 3:
        return np.mean(x, axis=(0, 1))

    if x.ndim == 4:
        return np.mean(x, axis=(1, 2))

    raise ValueError(f"Input harus 3D atau 4D, dapat shape {x.shape}")

```

2.1.8. GlobalMaxPooling2D

GlobalMaxPooling2D adalah layer pooling global from scratch yang mengambil nilai maksimum dari seluruh dimensi spasial.

2.1.8.1. Atribut

Kelas ini tidak memiliki atribut trainable.

2.1.8.2. Metode

forward(self, x)

Untuk input 3D atau 4D, method ini mengambil nilai maksimum pada dimensi tinggi dan lebar untuk setiap channel.

```
def forward(self, x):
    if x.ndim == 3:
        return np.max(x, axis=(0, 1))

    if x.ndim == 4:
        return np.max(x, axis=(1, 2))

    raise ValueError(f"Input harus 3D atau 4D, dapat shape {x.shape}")
```

2.1.9. LocallyConnected2D

LocallyConnected2D adalah implementasi non-shared convolution from scratch. Layer ini mirip dengan Conv2D, tetapi setiap posisi output memiliki kernel yang berbeda sehingga jumlah parameternya jauh lebih besar.

2.1.9.1. Atribut

weights

Bobot lokal dengan shape (output_rows * output_cols, kernel_height * kernel_width * input_channel, output_channel).

bias

Bias untuk setiap posisi output dan output channel.

kernel_size

Ukuran kernel lokal.

strides

Besar langkah perpindahan kernel.

padding

Jenis padding yang digunakan.

activation

Nama aktivasi yang diterapkan setelah operasi lokal.

2.1.9.2. Metode

__init__(self, weights, bias, kernel_size, strides=(1, 1), padding="valid", activation=None)

Menginisialisasi bobot lokal, bias, ukuran kernel, stride, padding, dan aktivasi.

```
def __init__(self, weights, bias, kernel_size, strides=(1, 1),
padding="valid", activation=None):
    self.weights = weights
    self.bias = bias
    self.kernel_size = kernel_size
    self.strides = strides
    self.padding = padding
    self.activation = activation
```

_pad_input(self, x)

Memberikan padding pada input sesuai mode yang dipilih.

```
def _pad_input(self, x):
    if self.padding == "valid":
        return x

    if self.padding == "same":
        _, h, w, _ = x.shape
        kh, kw = self.kernel_size
        sh, sw = self.strides

        out_h = int(np.ceil(h / sh))
        out_w = int(np.ceil(w / sw))

        pad_h = max((out_h - 1) * sh + kh - h, 0)
        pad_w = max((out_w - 1) * sw + kw - w, 0)

        pad_top = pad_h // 2
        pad_bottom = pad_h - pad_top
        pad_left = pad_w // 2
        pad_right = pad_w - pad_left

        return np.pad(
            x,
            (
                (0, 0),
                (pad_top, pad_bottom),
            )
```

```

        (pad_left, pad_right),
        (0, 0),
    ),
    mode="constant",
    constant_values=0,
)

raise ValueError(f"Padding tidak dikenal: {self.padding}")

```

forward(self, x)

Mengekstraksi patch input, mengubah patch menjadi shape (N, P, K), lalu menghitung output menggunakan `np.einsum("npk,pkf->npf", patches, weights)`. Hasilnya di-reshape kembali menjadi feature map.

```

def forward(self, x):
    is_single = False

    if x.ndim == 3:
        x = np.expand_dims(x, axis=0)
        is_single = True

    x_padded = self._pad_input(x)

    kh, kw = self.kernel_size
    sh, sw = self.strides

    windows = sliding_window_view(
        x_padded,
        window_shape=(kh, kw),
        axis=(1, 2)
    )

    windows = windows[:, ::sh, ::sw, :, :, :]

    # windows awal:
    # (N, out_h, out_w, C_in, kh, kw)
    # ubah ke:
    # (N, out_h, out_w, kh, kw, C_in)
    windows = np.moveaxis(windows, 3, -1)

```

```

n, out_h, out_w, _, _, _ = windows.shape
num_positions = out_h * out_w

# patches:
# (N, output_rows * output_cols, kh * kw * C_in)
patches = windows.reshape(n, num_positions, -1)

expected_positions = self.weights.shape[0]

if expected_positions != num_positions:
    raise ValueError(
        f"Jumlah posisi output tidak cocok. "
        f"weights punya {expected_positions} posisi, "
        f"input menghasilkan {num_positions} posisi. "
        f"Cek padding, strides, kernel_size, atau input
size."
    )

# patches: (N, P, K)
# weights: (P, K, C_out)
# output: (N, P, C_out)
output = np.einsum("npk,pkf->npf", patches, self.weights)

if self.bias is not None:
    output = output + self.bias

c_out = self.weights.shape[-1]
output = output.reshape(n, out_h, out_w, c_out)

output = apply_activation(output, self.activation)

if is_single:
    return output[0]

return output

```

2.1.10. ScratchSequential

ScratchSequential adalah container sederhana untuk menjalankan beberapa layer from scratch secara berurutan.

2.1.10.1. Atribut

layers

List layer yang memiliki method forward.

2.1.10.2. Metode

__init__(self, layers)

Menyimpan urutan layer yang akan dijalankan.

```
def __init__(self, layers):  
    self.layers = layers
```

forward(self, x)

Meneruskan input ke setiap layer secara berurutan. Output dari sebuah layer menjadi input untuk layer berikutnya.

```
def forward(self, x):  
    out = x  
  
    for layer in self.layers:  
        out = layer.forward(out)  
  
    return out
```

2.1.11. Embedding

Embedding adalah implementasi embedding layer from scratch untuk mengubah token caption menjadi vektor kontinu.

2.1.11.1. Atribut

weights

Matriks embedding dengan shape (vocab_size, embed_dim).

2.1.11.2. Metode

__init__(self, weights)

Menginisialisasi matriks embedding.

```
def __init__(self, weights):  
    """  
    weights: numpy array dengan shape (vocab_size, embed_dim)  
    """
```

```
self.weights = weights
```

forward(self, inputs)

Mengambil vektor embedding berdasarkan indeks token input.

```
def forward(self, inputs):  
    return self.weights[inputs]
```

2.1.12. SimpleRNNCell

SimpleRNNCell adalah unit recurrent dasar yang menghitung hidden state pada satu timestep.

2.1.12.1. Atribut

kernel

Bobot input-ke-hidden dengan shape (input_dim, units).

recurrent_kernel

Bobot hidden-ke-hidden dengan shape (units, units).

bias

Bias hidden state.

activation

Nama aktivasi recurrent, yaitu tanh pada eksperimen ini.

2.1.12.2. Metode

__init__(self, kernel, recurrent_kernel, bias, activation="tanh")

Menginisialisasi bobot input, bobot recurrent, bias, dan aktivasi.

```
def __init__(self, kernel, recurrent_kernel, bias,  
activation="tanh"):  
    self.kernel = kernel # shape:  
(input_dim, units)  
    self.recurrent_kernel = recurrent_kernel # shape: (units,  
units)  
    self.bias = bias # shape: (units,)   
    self.activation = activation
```

forward(self, x_t, h_prev)

Menghitung hidden state baru dengan persamaan activation.

```
def forward(self, x_t, h_prev):
    z = np.dot(x_t, self.kernel) + np.dot(h_prev,
self.recurrent_kernel) + self.bias
    h_t = apply_activation(z, self.activation)
    return h_t
```

2.1.13. SimpleRNN

SimpleRNN adalah wrapper untuk menjalankan SimpleRNNCell sepanjang sequence. Kelas ini digunakan untuk forward propagation sequence secara batch.

2.1.13.1. Atribut

cell

Objek SimpleRNNCell yang dipakai pada setiap timestep.

return_sequences

Menentukan apakah output berupa seluruh hidden state setiap timestep atau hanya hidden state terakhir.

units

Jumlah unit hidden state.

2.1.13.2. Metode

__init__(self, kernel, recurrent_kernel, bias, return_sequences=False, activation="tanh")

Membuat cell RNN dan menyimpan konfigurasi output.

```
def __init__(self, kernel, recurrent_kernel, bias,
return_sequences=False, activation="tanh"):
    self.cell = SimpleRNNCell(kernel, recurrent_kernel, bias,
activation)
    self.return_sequences = return_sequences
    self.units = recurrent_kernel.shape[0]
```

forward(self, inputs, initial_state=None)

Memproses input dengan shape (batch_size, seq_len, input_dim) dari timestep pertama sampai terakhir. Jika initial_state tidak diberikan, hidden state awal diinisialisasi nol.

```

def forward(self, inputs, initial_state=None):
    # shape: (batch_size, seq_len, input_dim)
    batch_size, seq_len, _ = inputs.shape

    if initial_state is None:
        h_t = np.zeros((batch_size, self.units))
    else:
        h_t = initial_state

    outputs = []
    for t in range(seq_len):
        h_t = self.cell.forward(inputs[:, t, :], h_t)
        outputs.append(h_t)

    if self.return_sequences:
        return np.stack(outputs, axis=1)
    else:
        return h_t

```

2.1.14. LSTMCell

LSTMCell adalah unit LSTM untuk satu timestep. Kelas ini mengimplementasikan mekanisme input gate, forget gate, candidate cell, dan output gate.

2.1.14.1. Atribut

kernel

Bobot input dengan shape (input_dim, 4 * units).

recurrent_kernel

Bobot recurrent dengan shape (units, 4 * units).

bias

Bias gabungan untuk empat gate.

units

Jumlah unit hidden state. Nilai ini diperoleh dari ukuran recurrent_kernel.

2.1.14.2. Metode

__init__(self, kernel, recurrent_kernel, bias)

Menginisialisasi bobot input, bobot recurrent, bias, dan jumlah unit.

```
def __init__(self, kernel, recurrent_kernel, bias):
    self.kernel = kernel
    self.recurrent_kernel = recurrent_kernel
    self.bias = bias
    self.units = recurrent_kernel.shape[0]
```

forward(self, x_t, h_prev, c_prev)

Menghitung nilai empat gate dengan urutan Keras, yaitu input, forget, cell, dan output. Method ini kemudian menghitung cell state baru dan hidden state baru.

```
def forward(self, x_t, h_prev, c_prev):
    z = np.dot(x_t, self.kernel) + np.dot(h_prev,
self.recurrent_kernel) + self.bias

    # urutan: input (i), forget (f), cell (c), output (o)
    i_gate = z[:, :self.units]
    f_gate = z[:, self.units:self.units*2]
    c_gate = z[:, self.units*2:self.units*3]
    o_gate = z[:, self.units*3:]

    i      = sigmoid(i_gate)
    f      = sigmoid(f_gate)
    c_can  = tanh(c_gate)
    o      = sigmoid(o_gate)

    c_t = f * c_prev + i * c_can
    h_t = o * tanh(c_t)

    return h_t, c_t
```

2.1.15. LSTM

LSTM adalah wrapper untuk menjalankan LSTMCell sepanjang sequence.

2.1.15.1. Atribut

cell

Objek LSTMCell yang dipakai pada setiap timestep.

return_sequences

Menentukan apakah output berupa seluruh hidden state setiap timestep atau hanya hidden state terakhir.

units

Jumlah unit hidden state.

2.1.15.2. Metode

__init__(self, kernel, recurrent_kernel, bias, return_sequences=False)

Membuat cell LSTM dan menyimpan konfigurasi output.

```
def __init__(self, kernel, recurrent_kernel, bias,
return_sequences=False):
    self.cell = LSTMCell(kernel, recurrent_kernel, bias)
    self.return_sequences = return_sequences
    self.units = recurrent_kernel.shape[0]
```

forward(self, inputs, initial_state=None)

Memproses sequence input dengan hidden state dan cell state. Jika state awal tidak diberikan, h_0 dan c_0 diinisialisasi nol.

```
def forward(self, inputs, initial_state=None):
    # shape: (batch_size, seq_len, input_dim)
    batch_size, seq_len, _ = inputs.shape

    # h_0 dan c_0 diinisialisasi zeros jika tidak ada dari
encoder
    if initial_state is None:
        h_t = np.zeros((batch_size, self.units))
        c_t = np.zeros((batch_size, self.units))
    else:
        h_t, c_t = initial_state

    outputs = []
    for t in range(seq_len):
        h_t, c_t = self.cell.forward(inputs[:, t, :], h_t, c_t)
        outputs.append(h_t)

    if self.return_sequences:
        return np.stack(outputs, axis=1), (h_t, c_t)
    else:
```

```
return h_t, (h_t, c_t)
```

2.1.16. ScratchDecoderInference

ScratchDecoderInference adalah kelas inference image captioning from scratch. Kelas ini memuat bobot dari decoder Keras, lalu melakukan greedy decoding dengan komponen NumPy.

2.1.16.1. Atribut

vocab

Mapping kata ke indeks token.

rev_vocab

Mapping indeks token ke kata.

rnn_type

Jenis recurrent decoder, yaitu RNN atau LSTM.

num_layers

Jumlah layer recurrent yang digunakan.

hidden_size

Ukuran hidden state setiap layer recurrent.

projection

Dense layer untuk memproyeksikan feature CNN 2048 dimensi ke embed_dim.

embedding

Embedding layer untuk token caption.

recurrent_cells

List cell recurrent yang berisi SimpleRNNCell atau LSTMCell.

dense_out

Dense softmax untuk menghasilkan distribusi probabilitas kata berikutnya.

2.1.16.2. Metode

__init__(self, vocab, rnn_type="RNN", num_layers=1, embed_dim=256, hidden_size=128)

Menginisialisasi struktur decoder scratch dengan bobot nol sementara.

```

def __init__(self, vocab, rnn_type="RNN", num_layers=1,
embed_dim=256, hidden_size=128):
    self.vocab = vocab
    self.rev_vocab = {v: k for k, v in vocab.items()}
    self.rnn_type = rnn_type
    self.num_layers = num_layers
    self.hidden_size = hidden_size

    vocab_size = len(vocab)

    self.projection = Dense(np.zeros((2048, embed_dim)),
np.zeros(embed_dim), activation=None)
    self.embedding = Embedding(np.zeros((vocab_size, embed_dim)))

    self.recurrent_cells = []
    for _ in range(num_layers):
        if rnn_type == "LSTM":
            self.recurrent_cells.append(LSTMCell(
                np.zeros((embed_dim if _ == 0 else hidden_size,
hidden_size * 4)),
                np.zeros((hidden_size, hidden_size * 4)),
                np.zeros(hidden_size * 4)
            ))
        else:
            self.recurrent_cells.append(SimpleRNNCell(
                np.zeros((embed_dim if _ == 0 else hidden_size,
hidden_size)),
                np.zeros((hidden_size, hidden_size)),
                np.zeros(hidden_size),
                activation="tanh"
            ))

    self.dense_out = Dense(np.zeros((hidden_size, vocab_size)),
np.zeros(vocab_size), activation="softmax")

```

load_weights_from_keras(self, keras_model)

Memuat bobot cnn_projection, caption_embedding, layer recurrent, dan dense_output dari model Keras ke komponen scratch.

```

def load_weights_from_keras(self, keras_model):

```

```

        proj_w, proj_b =
keras_model.get_layer("cnn_projection").get_weights()
        self.projection.weights = proj_w
        self.projection.bias = proj_b

        self.embedding.weights =
keras_model.get_layer("caption_embedding").get_weights()[0]

        for i in range(self.num_layers):
            layer_name = f"{'lstm' if self.rnn_type == 'LSTM' else
'rnn'}_layer_{i+1}"
            k_weights =
keras_model.get_layer(layer_name).get_weights()
            self.recurrent_cells[i].kernel = k_weights[0]
            self.recurrent_cells[i].recurrent_kernel = k_weights[1]
            self.recurrent_cells[i].bias = k_weights[2]

        out_w, out_b =
keras_model.get_layer("dense_output").get_weights()
        self.dense_out.weights = out_w
        self.dense_out.bias = out_b

```

generate_caption(self, cnn_feature, max_length=40)

Melakukan inference caption dengan greedy decoding. Feature CNN terlebih dahulu diproyeksikan sebagai input pre-inject, kemudian decoder mulai menghasilkan kata dari token <start> hingga menemukan token <end> atau mencapai batas panjang.

```

def generate_caption(self, cnn_feature, max_length=40):
    states = []
    for _ in range(self.num_layers):
        if self.rnn_type == "LSTM":
            states.append((np.zeros((1, self.hidden_size)),
np.zeros((1, self.hidden_size))))
        else:
            states.append(np.zeros((1, self.hidden_size)))

    # t = -1 (preinject)
    x_t = self.projection.forward(cnn_feature.reshape(1, -1))

    # forward w/o predicgt

```

```

        for i in range(self.num_layers):
            if self.rnn_type == "LSTM":
                h_t, c_t = self.recurrent_cells[i].forward(x_t,
states[i][0], states[i][1])
                states[i] = (h_t, c_t)
                x_t = h_t
            else:
                h_t = self.recurrent_cells[i].forward(x_t, states[i])
                states[i] = h_t
                x_t = h_t

        current_word_idx = self.vocab["<start>"]
        caption = []

        for _ in range(max_length):
            x_t =
self.embedding.forward(np.array([current_word_idx]))

            for i in range(self.num_layers):
                if self.rnn_type == "LSTM":
                    h_t, c_t = self.recurrent_cells[i].forward(x_t,
states[i][0], states[i][1])
                    states[i] = (h_t, c_t)
                    x_t = h_t
                else:
                    h_t = self.recurrent_cells[i].forward(x_t,
states[i])
                    states[i] = h_t
                    x_t = h_t

            # pred next word
            probs = self.dense_out.forward(x_t)
            next_word_idx = np.argmax(probs, axis=-1)[0]

            if next_word_idx == self.vocab["<end>"]:
                break

            word = self.rev_vocab.get(next_word_idx, "<unk>")
            caption.append(word)
            current_word_idx = next_word_idx

        return " ".join(caption)

```

2.2. Fungsi Pendukung

Selain kelas-kelas di atas, implementasi juga memuat fungsi pendukung untuk menjalankan pipeline eksperimen secara lengkap.

2.2.1. Fungsi Aktivasi

Fungsi aktivasi berada pada `src/common/activations.py`.

`relu(x)`

Menghasilkan $\max(0, x)$.

```
def relu(x):  
    return np.maximum(0, x)
```

`softmax(x)`

Mengubah logits menjadi distribusi probabilitas pada axis terakhir. Implementasi mengurangi nilai maksimum input terlebih dahulu untuk menjaga stabilitas numerik.

```
def softmax(x):  
    x_shifted = x - np.max(x, axis=-1, keepdims=True)  
    exp_x = np.exp(x_shifted)  
    return exp_x / np.sum(exp_x, axis=-1, keepdims=True)
```

`sigmoid(x)`

Menghitung fungsi sigmoid.

```
def sigmoid(x):  
    return 1 / (1 + np.exp(-x))
```

`tanh(x)`

Menghitung fungsi hyperbolic tangent.

```
def tanh(x):  
    return np.tanh(x)
```

apply_activation(x, activation=None)

Memilih dan menerapkan fungsi aktivasi berdasarkan nama aktivasi.

```
def apply_activation(x, activation=None):
    if activation is None:
        return x

    if activation == "relu":
        return relu(x)

    if activation == "softmax":
        return softmax(x)

    if activation == "sigmoid":
        return sigmoid(x)

    if activation == "tanh":
        return tanh(x)

    raise ValueError(f"Activation tidak dikenal: {activation}")
```

2.2.2. Utility Pemrosesan Gambar

Fungsi utility gambar berada pada src/common/image_utils.py.

load_image(image_path, target_size=(150, 150), normalize_range="0_1")

Membaca satu gambar, mengubahnya ke RGB, melakukan resize, mengubahnya menjadi array NumPy, dan menormalisasi piksel.

load_images(image_paths, target_size=(150, 150), normalize_range="0_1")

Memproses sekumpulan gambar menjadi batch array dengan shape (N, H, W, C).

extract_features(image_paths, output_path, model_name="InceptionV3", batch_size=32, force=False)

Mengekstraksi feature vector menggunakan CNN encoder pretrained Keras yang dibekukan, lalu menyimpan setiap feature ke file .npy.

load_feature(npy_path)

Memuat satu feature vector dari file .npy.

load_features_from_dir(feature_dir, image_names=None)

Memuat seluruh atau sebagian feature vector dari direktori.

list_image_paths_by_class(root_dir)

Membaca dataset klasifikasi gambar berdasarkan struktur folder kelas dan mengembalikan path gambar, label integer, dan nama kelas.

2.2.3. Konfigurasi Eksperimen CNN

Fungsi konfigurasi berada pada src/cnn/config.py.

get_all_configs()

Menghasilkan 16 konfigurasi CNN dari kombinasi jumlah layer konvolusi, jumlah filter, ukuran kernel, dan jenis pooling.

get_config_by_name(name)

Mengambil konfigurasi CNN berdasarkan nama konfigurasi.

2.2.4. Builder Model CNN Keras

Fungsi builder model berada pada src/cnn/keras_models.py.

build_baseline_cnn(input_shape=(150, 150, 3), num_classes=6)

Membangun baseline CNN dengan Conv2D shared parameter.

build_local_cnn(input_shape=(150, 150, 3), num_classes=6)

Membangun baseline CNN non-shared parameter menggunakan KerasLocallyConnected2D.

build_cnn_from_config(config, input_shape=(150, 150, 3), num_classes=6)

Membangun model CNN shared parameter berdasarkan konfigurasi eksperimen.

build_local_cnn_from_config(config, input_shape=(150, 150, 3), num_classes=6)

Membangun model CNN non-shared parameter dengan arsitektur yang ekuivalen terhadap konfigurasi CNN shared.

2.2.5. Training dan Evaluasi CNN

Fungsi training dan evaluasi berada pada src/cnn/training.py.

prepare_datasets(train_dir, test_dir, img_size=(150, 150), batch_size=32, val_split=0.2, seed=42)

Membuat dataset train, validation, dan test dari folder gambar, melakukan normalisasi piksel ke rentang [0, 1], dan menambahkan prefetch.

train_single_model(config, train_ds, val_ds, epochs=10, ...)

Melatih satu model CNN berdasarkan konfigurasi, menyimpan model, bobot, dan history. Jika file model dan history sudah ada serta force=False, training dilewati.

evaluate_model(model, test_ds, class_names=None)

Melakukan prediksi pada test set dan menghitung macro F1-score, accuracy, classification report, confusion matrix, serta jumlah parameter.

save_evaluation(result, config, results_dir="results/cnn")

Menyimpan hasil evaluasi ke file JSON dan menyimpan y_true serta y_pred ke file .npy.

run_all_experiments(configs, train_ds, val_ds, test_ds, ...)

Menjalankan semua konfigurasi eksperimen CNN, mengevaluasi setiap model, dan membuat summary ranking berdasarkan macro F1-score.

find_best_config(all_results)

Memilih konfigurasi terbaik berdasarkan macro F1-score tertinggi.

train_locally_connected(config, train_ds, val_ds, ...)

Melatih model non-shared parameter berdasarkan konfigurasi terbaik.

load_all_results(configs, results_dir="results/cnn")

Memuat metrik dan history eksperimen dari disk.

2.2.6. Loader Bobot CNN

Fungsi loader bobot berada pada src/cnn/weight_loader_cnn.py.

build_scratch_from_keras(keras_model_or_path, config)

Membangun ScratchSequential shared parameter dari model Keras dengan memuat bobot Conv2D, pooling, Flatten atau global pooling, serta Dense layer.

build_scratch_local_from_keras(keras_model_or_path, config)

Membangun ScratchSequential non-shared parameter dari model Keras yang memakai KerasLocallyConnected2D.

build_scratch_from_keras_model(keras_model_path)

Helper untuk membangun scratch model baseline dari path model Keras.

2.2.7. Preprocessing Caption

Fungsi preprocessing caption berada pada src/rnn_lstm/caption_preprocess.py.

clean_caption(caption)

Membersihkan caption dengan lowercase, menghapus karakter selain huruf dan spasi, merapikan whitespace, serta menambahkan token <start> dan <end>.

```
# build_vocabulary(captions, vocab_threshold=5)
```

Membangun vocabulary dari caption training. Vocabulary selalu memuat token <pad>, <start>, <end>, dan <unk>.

```
# preprocess_captions(raw_captions_dict, vocab, max_len)
```

Mengubah caption menjadi sequence integer, membuat sequence input dan target, serta melakukan padding ke panjang seragam.

2.2.8. Feature Extraction RNN/LSTM

Fungsi feature extraction berada pada src/rnn_lstm/feature_extractor.py.

```
# extract_and_save_features(image_dir, output_dir, model_name="InceptionV3",  
batch_size=32, force=False)
```

Mengekstraksi feature vector untuk seluruh gambar pada direktori menggunakan CNN encoder pretrained dan menyimpan hasilnya ke file .npy.

2.2.9. Decoder Keras RNN/LSTM

Fungsi decoder Keras berada pada src/rnn_lstm/scratch_decoder.py.

```
# build_decoder_model(vocab_size, max_seq_len, embed_dim, rnn_type, num_layers,  
hidden_size)
```

Membangun decoder image captioning dengan arsitektur pre-inject. Feature CNN diproyeksikan ke embed_dim, di-concatenate sebelum embedding token caption, lalu diproses oleh satu atau beberapa layer SimpleRNN atau LSTM. Output akhirnya berupa distribusi softmax pada vocabulary.

2.2.10. Evaluasi dan Inference Caption

Fungsi evaluasi caption berada pada src/rnn_lstm/inference_utils.py.

```
# evaluate_model(model, X_cnn_test, y_captions_test_raw, max_length=40)
```

Mengevaluasi scratch decoder dengan BLEU-4, METEOR, waktu eksekusi, dan hasil detail setiap gambar.

```
# generate_caption_keras(keras_model, cnn_feature, vocab, rev_vocab,  
model_max_seq_len, limit_length)
```

Melakukan greedy decoding menggunakan model Keras.

```
# evaluate_model_keras(keras_model, X_cnn_test, y_captions_test_raw, vocab,  
model_max_seq_len, limit_length=40)
```

Mengevaluasi decoder Keras dengan metrik BLEU-4, METEOR, waktu eksekusi, dan hasil detail setiap gambar.

2.3. Alur Implementasi

2.3.1. Alur CNN

Alur implementasi CNN dimulai dari pembacaan dataset Intel Image Classification dengan struktur folder kelas. Dataset dibagi menjadi train, validation, dan test. Setelah itu, 16 konfigurasi CNN dibuat dari variasi jumlah layer konvolusi, jumlah filter, ukuran kernel, dan jenis pooling.

Setiap konfigurasi dilatih menggunakan Keras dengan Conv2D shared parameter. Model, bobot, history, prediksi, dan metrik evaluasi disimpan ke disk. Konfigurasi terbaik dipilih berdasarkan macro F1-score. Setelah konfigurasi terbaik diperoleh, model non-shared parameter dilatih dengan mengganti Conv2D menjadi KerasLocallyConnected2D. Bobot model Keras kemudian dimuat ke implementasi from scratch untuk membandingkan forward propagation Keras dan NumPy.

2.3.2. Alur RNN/LSTM

Alur implementasi RNN/LSTM dimulai dari ekstraksi feature vector gambar Flickr8k menggunakan CNN encoder pretrained. Caption dibersihkan, diberi token khusus, diubah menjadi indeks vocabulary, dan dipadding ke panjang seragam.

Decoder Keras dibangun dengan metode pre-inject, yaitu feature CNN diproyeksikan menjadi timestep pertama sebelum sequence caption. Decoder dilatih untuk RNN dan LSTM dengan variasi jumlah layer recurrent dan ukuran hidden state. Setelah training, bobot decoder Keras dimuat ke scratch decoder. Caption dihasilkan dengan greedy decoding dan dievaluasi menggunakan BLEU-4, METEOR, serta waktu eksekusi.

BAB III

PENJELASAN FORWARD PROPAGATION

Forward propagation adalah proses menghitung output model dari input tanpa melakukan pembaruan bobot. Pada tugas ini, forward propagation from scratch diimplementasikan menggunakan NumPy. Bobot yang digunakan pada implementasi scratch berasal dari model Keras yang sudah dilatih. Dengan demikian, output model scratch dapat dibandingkan langsung dengan output Keras untuk memastikan bahwa implementasi *forward propagation* sudah ekuivalen.

Implementasi *forward propagation* pada program ini terbagi menjadi tiga bagian: CNN untuk klasifikasi gambar, RNN untuk image captioning, dan LSTM untuk image captioning. CNN menerima input gambar dan menghasilkan probabilitas kelas. RNN dan LSTM menerima vektor fitur gambar dari CNN encoder serta sequence token caption, lalu menghasilkan distribusi probabilitas kata berikutnya.

3.1. CNN

Forward propagation CNN digunakan pada task image classification Intel Image Classification. Input model berupa gambar RGB yang sudah di-*resize* ke ukuran 150 x 150 dan dinormalisasi ke rentang [0, 1]. Untuk satu gambar, bentuk input adalah (150, 150, 3). Untuk batch berisi N gambar, bentuk input adalah (N , 150, 150, 3).

Arsitektur CNN yang digunakan pada eksperimen dibangun dari blok:

1. Conv2D atau LocallyConnected2D.
2. ReLU.
3. MaxPooling2D atau AveragePooling2D.
4. Flatten.
5. Dense 128 dengan ReLU.
6. Dense output dengan softmax.

Untuk model *shared parameter*, layer konvolusi menggunakan Conv2D. Untuk model *non-shared parameter*, layer konvolusi diganti dengan LocallyConnected2D. *Forward propagation* dari model scratch dijalankan oleh ScratchSequential, yaitu container yang meneruskan output dari satu layer sebagai input ke layer berikutnya.

Untuk satu batch gambar, alur forward propagation CNN adalah:

1. Input gambar masuk sebagai tensor (N , 150, 150, 3).
2. Setiap blok konvolusi menghitung feature map dengan Conv2D atau LocallyConnected2D.
3. ReLU menghilangkan nilai negatif pada feature map.
4. Max pooling atau average pooling mereduksi dimensi spasial.
5. Flatten mengubah feature map menjadi vektor.
6. Dense 128 ReLU membentuk representasi fitur tingkat tinggi.

7. Dense output softmax menghasilkan probabilitas enam kelas.
8. Kelas prediksi dipilih menggunakan argmax.

Karena bobot scratch dimuat dari model Keras, hasil probabilitas scratch seharusnya sangat dekat dengan hasil probabilitas Keras. Perbedaan kecil masih mungkin terjadi karena detail presisi floating point.

3.1.1. Persiapan Input

Sebelum masuk ke model, gambar dibaca menggunakan `load_image`. Gambar dibuka dengan Pillow, dikonversi ke RGB, di-resize, lalu diubah menjadi array NumPy bertipe float32. Nilai piksel dibagi 255 sehingga berada pada rentang [0, 1].

Jika input hanya satu gambar dengan shape (H, W, C) , beberapa layer scratch akan menambahkan dimensi batch sehingga bentuknya menjadi $(1, H, W, C)$. Setelah perhitungan selesai, dimensi batch ini dihapus kembali agar output tetap konsisten untuk kasus single image.

3.1.2. Forward Propagation Conv2D Shared Parameter

Pada Conv2D shared parameter, satu kernel yang sama digunakan untuk seluruh posisi spasial pada gambar. Jika input batch ditulis sebagai X dengan shape (N, H, W, C_{in}) dan bobot kernel ditulis sebagai W_{conv} dengan shape $(K_h, K_w, C_{in}, C_{out})$, maka output konvolusi memiliki shape $(N, H_{out}, W_{out}, C_{out})$.

Untuk setiap gambar n , posisi output (i, j) , dan filter output f , nilai konvolusi dihitung sebagai:

$$Z[n, i, j, f] = \sum_{a=0}^{K_h-1} \sum_{b=0}^{K_w-1} \sum_{c=0}^{C_{in}-1} X_{padded}[n, i_s + a, j_s + b, c] \times W_{conv}[a, b, c, f] + bias[f]$$

dengan:

$$i_s = i \times stride_h$$

$$j_s = j \times stride_w$$

Pada implementasi, operasi tersebut tidak ditulis sebagai loop tiga tingkat untuk setiap patch. Program menggunakan "sliding_window_view" untuk membentuk seluruh patch input sekaligus. Setelah patch terbentuk, axis channel dipindahkan agar bentuk patch konsisten dengan bentuk kernel Keras:

$$(N, H_{out}, W_{out}, K_h, K_w, C_{in})$$

Kemudian seluruh dot product antara patch dan kernel dihitung. Operasi tersebut menjumlahkan dimensi K_h , K_w , dan C_{in} , sehingga tersisa dimensi batch, tinggi output, lebar output, dan channel output.

Setelah operasi linear selesai, bias ditambahkan pada setiap output channel:

$$Z = Z + bias$$

Kemudian aktivasi diterapkan. Pada layer konvolusi eksperimen ini, aktivasi yang digunakan adalah ReLU:

$$A = ReLU(Z) = \max(0, Z)$$

3.1.3. Padding pada Conv2D

Implementasi Conv2D mendukung padding valid dan same. Pada padding valid, input tidak diberi padding. Akibatnya, ukuran output akan lebih kecil jika ukuran kernel lebih besar dari 1. Pada padding same, input diberi padding nol di sisi atas, bawah, kiri, dan kanan agar ukuran output mengikuti ukuran input terhadap stride. Untuk tinggi input H , lebar input W , ukuran kernel (K_h, K_w) , dan stride (S_h, S_w) , ukuran output dihitung sebagai:

$$H_{out} = \text{ceil}\left(\frac{H}{S_h}\right)$$

$$W_{out} = \text{ceil}\left(\frac{W}{S_w}\right)$$

Jumlah padding total dihitung dengan:

$$pad_h = \max\left((H_{out} - 1) \times S_h + K_h - H, 0\right)$$

$$pad_w = \max\left((W_{out} - 1) \times S_w + K_w - W, 0\right)$$

Padding kemudian dibagi ke sisi atas-bawah dan kiri-kanan. Jika jumlah padding ganjil, sisa padding diletakkan pada sisi bawah atau kanan. Cara ini mengikuti perilaku umum padding same.

3.1.4. Forward Propagation LocallyConnected2D Non-Shared Parameter

LocallyConnected2D memiliki pola perhitungan yang mirip dengan Conv2D, tetapi tidak menggunakan parameter sharing. Artinya, setiap posisi output memiliki kernel yang berbeda.

Jika terdapat $P = H_{out} \times W_{out}$ posisi output, maka bobot layer memiliki shape $(P, K_h \times K_w \times C_{in}, C_{out})$. Setelah patch input diekstraksi, patch diubah menjadi bentuk (N, P, K) dengan $K = K_h \times K_w \times C_{in}$.

Untuk setiap gambar n , posisi output p , dan output channel f , perhitungan linear dilakukan sebagai:

$$Z[n, p, f] = \sum_{k=0}^{K-1} patches[n, p, k] \times W_{local}[p, k, f] + bias[p, f]$$

Setelah output (N, P, C_{out}) diperoleh, output di-*reshape* kembali menjadi $(N, H_{out}, W_{out}, C_{out})$. Kemudian bias dan aktivasi diterapkan. Pada model *non-shared* yang digunakan, aktivasi setelah LocallyConnected2D adalah ReLU.

Perbedaan utama antara Conv2D dan LocallyConnected2D berada pada bobot yang digunakan. Conv2D memakai satu kernel untuk semua posisi, sedangkan LocallyConnected2D memakai kernel berbeda untuk setiap posisi. Akibatnya, LocallyConnected2D memiliki jumlah parameter jauh lebih besar.

3.1.5. Forward Propagation Pooling

Setelah layer konvolusi, model menggunakan pooling untuk mereduksi ukuran spasial feature map. Pooling tidak memiliki parameter trainable.

Pada MaxPooling2D, setiap window berukuran (P_h, P_w) diganti dengan nilai maksimum pada window tersebut:

$$Y[n, i, j, c] = \max(windowX[n, i_s:i_s + P_h, j_s:j_s + P_w, c])$$

Pada AveragePooling2D, setiap window diganti dengan nilai rata-rata:

$$Y[n, i, j, c] = \text{mean}(windowX[n, i_s:i_s + P_h, j_s:j_s + P_w, c])$$

Implementasi pooling juga menggunakan "sliding_window_view". Window yang terbentuk kemudian diambil setiap stride:

$$windows = windows[:, ::sh, ::sw, :, :, :]$$

Pooling mempertahankan jumlah channel, tetapi mengurangi tinggi dan lebar feature map.

3.1.6. Forward Propagation Flatten dan Dense

Setelah seluruh blok konvolusi dan pooling selesai, feature map diubah menjadi vektor menggunakan Flatten. Untuk input batch dengan shape:

$$(N, H, W, C)$$

Flatten menghasilkan:

$$(N, H \times W \times C)$$

Hasil Flatten kemudian masuk ke Dense layer. Dense layer pertama menggunakan 128 unit dan aktivasi ReLU:

$$\bar{Z}_1 = \bar{x}_{flat} \cdot W_1 + b_1$$

$$\bar{A}_1 = ReLU(\bar{Z}_1)$$

Dense layer terakhir menghasilkan enam nilai logit, satu untuk setiap kelas Intel Image Classification:

$$\bar{Z}_2 = \bar{A}_1 \cdot W_2 + b_2$$

Logit tersebut diubah menjadi probabilitas menggunakan softmax:

$$softmax(z_i) = \frac{e^{z_i - \max(\bar{z})}}{\sum_j e^{z_j - \max(\bar{z})}}$$

Pengurangan $\max(\bar{z})$ dilakukan untuk menjaga stabilitas numerik. Prediksi kelas akhir diperoleh dengan mengambil indeks probabilitas terbesar menggunakan $\text{argmax}(\text{softmax}(Z_2))$.

3.2. RNN

Forward propagation RNN digunakan pada task image captioning. Output modelnya adalah sequence kata yang mendeskripsikan gambar (caption).

Arsitektur captioning menggunakan metode pre-inject. Pada metode ini, feature vector gambar dari CNN encoder diproyeksikan ke dimensi embedding, kemudian dimasukkan sebagai input *time step* pertama sebelum token <start>.

Alur forward propagation implementasi RNN pada inference adalah:

1. Inisialisasi hidden state setiap layer dengan nol.
2. Proyeksikan feature CNN 2048 dimensi menjadi vector embed_dim.
3. Jalankan timestep pre-inject untuk memperbarui hidden state awal berdasarkan informasi gambar.
4. Tetapkan token awal sebagai <start>.
5. Ambil embedding dari token saat ini.
6. Jalankan embedding melalui seluruh layer RNN.
7. Hitung probabilitas kata berikutnya dengan Dense output dan softmax.
8. Ambil token dengan probabilitas terbesar.
9. Jika token adalah <end>, berhenti.
10. Jika bukan, tambahkan kata ke caption dan ulangi sampai mencapai max_length.

Dengan alur ini, informasi gambar dimasukkan ke hidden state sebelum kata pertama diprediksi. Caption yang dihasilkan kemudian bergantung pada feature gambar dan seluruh kata yang telah diprediksi sebelumnya.

3.2.1. Input dan Representasi Data

Input utama decoder terdiri atas:

1. Feature vector gambar hasil CNN encoder pretrained. Pada implementasi ini, shape feature adalah (2048,).
2. Sequence token caption dalam bentuk indeks vocabulary. Saat training, sequence ini digunakan dengan teacher forcing. Saat inference, token yang digunakan berasal dari prediksi sebelumnya.
3. Vocabulary, yaitu mapping kata ke indeks token, termasuk token khusus <pad>, <start>, <end>, dan <unk>.

Pada model Keras, feature CNN memiliki input (*batch_size*, 2048), caption input memiliki shape (*batch_size*, *max_seq_len* - 1)

Feature CNN diproyeksikan menjadi (*batch_size*, *embed_dim*). Setelah reshape, feature tersebut menjadi satu timestep (*batch_size*, 1, *embed_dim*).

Caption token diubah oleh embedding layer menjadi:

$$(batch_size, max_seq_len - 1, embed_dim)$$

Kemudian keduanya digabungkan pada axis waktu:

$$(batch_size, max_seq_len, embed_dim)$$

Sequence inilah yang menjadi input recurrent layer.

3.2.2. Proyeksi Feature CNN

Feature CNN tidak langsung dimasukkan ke RNN karena dimensinya 2048, sedangkan input recurrent layer menggunakan *embed_dim*. Oleh karena itu feature CNN diproyeksikan menggunakan Dense layer:

$$x_{-1} = cnn_feature \cdot W_{proj} + b_{proj}$$

x_{-1} adalah timestep pre-inject, input ini dimasukkan sebelum token <start>. Hasil proyeksi menjadi input pertama untuk recurrent cell. Hidden state awal diinisialisasi nol.

Pada kode inference, setelah feature diproyeksikan, feature tersebut langsung diproses oleh setiap layer recurrent untuk memperbarui hidden state awal sebelum proses decoding token dimulai.

3.2.3. Embedding Token Caption

Setiap token kata direpresentasikan sebagai indeks integer. Indeks tersebut tidak langsung diproses oleh RNN, tetapi terlebih dahulu diubah menjadi embedding vector $x_t = E[token_t]$, dengan E adalah matriks embedding dengan shape $(vocab_size, embed_dim)$.

Jika input token memiliki shape $(batch_size,)$, maka output embedding memiliki shape $(batch_size, embed_dim)$.

3.2.4. Forward Propagation

RNN menyimpan satu hidden state. Pada setiap timestep t, input embedding x_t dan hidden state sebelumnya h_{t-1} digunakan untuk menghitung hidden state baru h_t .

Persamaan forward propagation RNN adalah:

$$z_t = x_t \cdot W_{xh} + h_{t-1} \cdot W_{hh} + b_h$$

$$h_t = \tanh(z_t).$$

Aktivasi yang digunakan adalah tanh. Fungsi tanh membatasi nilai hidden state pada rentang $[-1, 1]$, sehingga hidden state dapat dipandang sebagai ringkasan informasi dari token-token sebelumnya.

Jika decoder memiliki lebih dari satu layer recurrent, output hidden state dari layer ke-i menjadi input untuk layer ke-i+1 pada timestep yang sama.

3.2.7. Dense Output dan Softmax

Setelah hidden state terakhir pada timestep tertentu diperoleh, model menghitung distribusi probabilitas atas seluruh vocabulary menggunakan Dense layer output:

$$\text{logits}_t = \text{dot}(h_t, W_{\text{out}}) + b_{\text{out}} \quad p_t = \text{softmax}(\text{logits}_t)$$

$$\text{logits}_t = h_t \cdot W_{\text{out}} + b_{\text{out}}$$

$$p_t = (\text{logits}_t)$$

Untuk inference, token berikutnya dipilih menggunakan greedy decoding $\text{argmax}(p_t)$. Jika next_token adalah <end>, proses decoding berhenti. Jika tidak, token tersebut ditambahkan ke caption dan digunakan sebagai input timestep berikutnya.

3.3. LSTM

LSTM digunakan pada task yang sama dengan RNN, yaitu image captioning. Perbedaannya adalah LSTM memiliki dua state: hidden state h_t dan cell state c_t . Cell state berfungsi sebagai jalur memori jangka panjang, sedangkan hidden state menjadi representasi output pada timestep saat ini.

Arsitektur input LSTM sama dengan RNN pada tugas ini. Feature CNN tetap dimasukkan menggunakan metode pre-inject, token caption tetap diubah menjadi embedding, dan output akhir tetap berupa distribusi softmax pada vocabulary.

Alur forward propagation LSTM pada inference adalah:

1. Inisialisasi hidden state dan cell state setiap layer dengan nol.
2. Proyeksikan feature CNN menjadi vector `embed_dim`.
3. Jalankan timestep pre-inject melalui semua layer LSTM untuk memasukkan informasi gambar ke state decoder.
4. Mulai decoding dari token `<start>`.
5. Ambil embedding token saat ini.
6. Jalankan embedding melalui seluruh layer LSTM.
7. Gunakan hidden state layer terakhir sebagai input Dense output.
8. Hitung probabilitas kata berikutnya dengan softmax.
9. Ambil token dengan probabilitas terbesar.
10. Berhenti jika token adalah `<end>` atau panjang caption mencapai `max_length`.

Perbedaan utama dari RNN adalah adanya cell state. Dengan cell state, informasi penting dari gambar dan token-token sebelumnya dapat dipertahankan lebih lama. Hal ini membantu LSTM menghasilkan caption yang lebih konsisten pada sequence yang panjang.

3.3.1. State pada LSTM

Pada setiap timestep, LSTM menerima tiga input utama berupa input saat ini x_t , hidden state sebelumnya h_{t-1} , dan cell state sebelumnya c_{t-1} . Kemudian LSTM menghasilkan dua state baru berupa hidden state baru h_t dan cell state baru c_t .

Pada awal sequence, jika tidak ada state yang diberikan, implementasi menginisialisasi keduanya dengan nol.

3.3.3. Input Gate

Input gate menentukan seberapa besar informasi baru akan ditulis ke cell state. Nilainya dihitung dengan fungsi sigmoid:

$$i_t = \text{sigmoid}(x_t \cdot W_{xi} + h_{t-1} \cdot W_{hi} + b_i)$$

Karena sigmoid menghasilkan nilai pada rentang $[0, 1]$, input gate bertindak seperti pengatur intensitas. Nilai mendekati 0 berarti informasi baru diabaikan, sedangkan nilai mendekati 1 berarti informasi baru diterima.

3.3.4. Forget Gate

Forget gate menentukan seberapa besar informasi lama pada cell state akan dipertahankan:

$$f_t = \text{sigmoid}(x_t \cdot W_{xf} + h_{t-1} \cdot W_{hf} + b_f)$$

Jika f_t mendekati 1, memori lama dipertahankan. Jika f_t mendekati 0, memori lama dihapus. Gate ini menjadi salah satu alasan LSTM lebih mampu menangani dependensi jangka panjang.

3.3.5. Candidate Cell

Candidate cell adalah kandidat informasi baru yang dapat dimasukkan ke cell state:

$$\hat{C}_t = \tanh(x_t \cdot W_{xc} + h_{t-1} \cdot W_{hc} + b_c)$$

Fungsi tanh menghasilkan nilai pada rentang $[-1, 1]$. Nilai ini belum langsung menjadi memori baru, tetapi terlebih dahulu dikalikan dengan input gate.

3.3.6. Pembaruan Cell State

Cell state baru dihitung dari kombinasi memori lama dan kandidat memori baru:

$$C_t = (C_{t-1} \odot f_t) \oplus (i_t \odot \hat{C}_t)$$

Bagian pertama, $C_{t-1} \odot f_t$ menentukan informasi lama yang dipertahankan. Bagian kedua, $i_t \odot \hat{C}_t$ menentukan informasi baru yang ditambahkan. Operasi perkalian dilakukan elemen per elemen (Hadamard Product).

3.3.7. Output Gate dan Hidden State

Output gate menentukan bagian cell state yang akan dikeluarkan sebagai hidden state:

$$o_t = \text{sigmoid}(x_t \cdot W_{xo} + h_{t-1} \cdot W_{ho} + b_o)$$

$$h_t = o_t \odot \tanh(C_t)$$

Hidden state h_t digunakan sebagai output LSTM pada timestep tersebut. Pada decoder captioning, hidden state ini kemudian diteruskan ke layer LSTM berikutnya atau ke Dense output untuk menghasilkan distribusi vocabulary.

3.3.9. Multi-layer LSTM

Pada LSTM dengan lebih dari satu layer, setiap layer memiliki pasangan state sendiri. Output hidden state dari layer sebelumnya menjadi input untuk layer berikutnya. Untuk layer pertama, inputnya adalah embedding token atau feature pre-inject.

Pada implementasi scratch, state setiap layer disimpan dalam list. Untuk LSTM, setiap elemen list adalah pasangan (h, c). Setelah sebuah layer memproses input, output h_t menjadi input untuk layer berikutnya.

BAB IV HASIL PENGUJIAN

4.1. CNN

Pengujian CNN dilakukan pada dataset Intel Image Classification yang terdiri atas enam kelas, yaitu buildings, forest, glacier, mountain, sea, dan street. Dataset uji berjumlah 3000 gambar. Seluruh model dievaluasi menggunakan macro F1-score karena metrik ini menghitung rata-rata performa tiap kelas tanpa terlalu didominasi oleh kelas dengan jumlah data yang lebih besar.

Eksperimen CNN dibagi menjadi tiga bagian utama:

1. Eksperimen 16 variasi arsitektur Conv2D shared parameter.
2. Perbandingan shared parameter dan non-shared parameter.
3. Perbandingan forward propagation Keras dan implementasi from scratch.

Seluruh model Conv2D dilatih selama 10 epoch. Konfigurasi eksperimen dibentuk dari kombinasi jumlah layer konvolusi, jumlah filter, ukuran kernel, dan jenis pooling. Hasil lengkap setiap eksperimen disimpan pada folder results/cnn.

4.1.1. Ringkasan Hasil 16 Eksperimen Conv2D

No	Konfigurasi	Conv Layer	Filter	Kernel	Pooling	Macro F1	Accuracy	Parameter
1	conv2_filters_32-64_kernels_3-3_pool_max	2	32-64	3-3	Max	0.7663	0.7640	11,235,142
2	conv2_filters_32-64_kernels_3-3_pool_average	2	32-64	3-3	Average	0.7561	0.7533	11,235,142
3	conv2_filters_32-64_kernels_5-5	2	32-64	5-5	Max	0.7364	0.7347	11,269,446

	5_pool_max							
4	conv2_filters_32-64_kernels_5-5_pool_average	2	32-64	5-5	Average	0.7144	0.7117	11,269,446
5	conv2_filters_64-128_kernels_3-3_pool_max	2	64-128	3-3	Max	0.7764	0.7750	22,506,246
6	conv2_filters_64-128_kernels_3-3_pool_average	2	64-128	3-3	Average	0.7308	0.7287	22,506,246
7	conv2_filters_64-128_kernels_5-5_pool_max	2	64-128	5-5	Max	0.7565	0.7537	22,640,390
8	conv2_filters_64-128_kernels_5-5_pool_average	2	64-128	5-5	Average	0.6756	0.6767	22,640,390
9	conv3_filters_32-64-128_kernels_3-3-3_pool_max	3	32-64-128	3-3-3	Max	0.7897	0.7900	5,402,566

	ls_3-3-3_pool_max							
10	conv3_filters_32-64-128_kernels_3-3-3_pool_average	3	32-64-128	3-3-3	Average	0.7890	0.7860	5,402,566
11	conv3_filters_32-64-128_kernels_5-5-5_pool_max	3	32-64-128	5-5-5	Max	0.7862	0.7850	5,567,942
12	conv3_filters_32-64-128_kernels_5-5-5_pool_average	3	32-64-128	5-5-5	Average	0.7618	0.7620	5,567,942
13	conv3_filters_64-128-256_kernels_3-3-3_pool_max	3	64-128-256	3-3-3	Max	0.7834	0.7820	10,988,550
14	conv3_filters_64-128-256_kernels_3-3-3_pool_average	3	64-128-256	3-3-3	Average	0.7693	0.7673	10,988,550
15	conv3_filters_64-128-256_kernels_5-5-5_pool_max	3	64-128-256	5-5-5	Max	0.7887	0.7857	11,646,

	4-128-256_kernels_5-5-5_pool_max		256					982
16	conv3_filters_64-128-256_kernels_5-5-5_pool_average	3	64-128-256	5-5-5	Average	0.7734	0.7727	11,646,982

Model terbaik berdasarkan macro F1-score adalah conv3_filters_32-64-128_kernels_3-3-3_pool_max, dengan macro F1-score sebesar 0.7897 dan accuracy sebesar 0.7900. Model ini menggunakan tiga layer konvolusi, filter 32-64-128, kernel 3x3 pada setiap layer, dan max pooling.

4.1.2. Analisis Model Terbaik

Model terbaik memiliki 5,402,566 parameter. Hasil classification report pada test set adalah sebagai berikut.

Kelas	Precision	Recall	F1-score	Support
buildings	0.7834	0.7117	0.7458	437
forest	0.9376	0.9198	0.9286	474
glacier	0.8441	0.6166	0.7126	553
mountain	0.7780	0.7410	0.7590	525
sea	0.6965	0.8686	0.7731	510
street	0.7525	0.8982	0.8189	501
macro avg	0.7987	0.7927	0.7897	3000

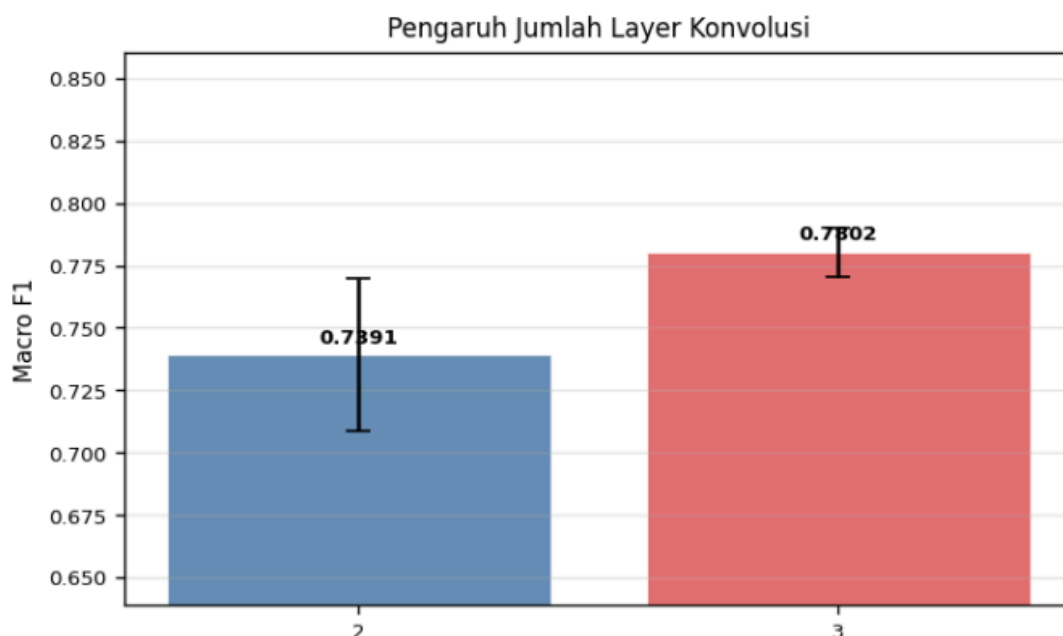
Berdasarkan tabel tersebut, kelas forest memiliki performa paling tinggi dengan F1-score 0.9286. Hal ini menunjukkan bahwa fitur visual pada kelas hutan relatif lebih mudah dipisahkan dari kelas lain. Kelas glacier memiliki recall paling rendah, yaitu 0.6166. Dari confusion matrix, banyak gambar glacier diprediksi sebagai mountain dan sea. Hal ini masuk akal karena ketiga kelas tersebut sering memiliki tekstur, warna, dan latar alam yang mirip.

Kurva training dan validation loss model terbaik menunjukkan bahwa training loss turun dari 0.9344 menjadi 0.0661. Validation loss terbaik terjadi pada epoch ke-2, yaitu 0.5686, lalu cenderung meningkat pada epoch berikutnya hingga 1.3479 pada epoch ke-10. Pola ini menunjukkan adanya overfitting setelah beberapa epoch awal. Meskipun begitu, performa test model ini tetap menjadi yang terbaik di antara 16 konfigurasi yang diuji.

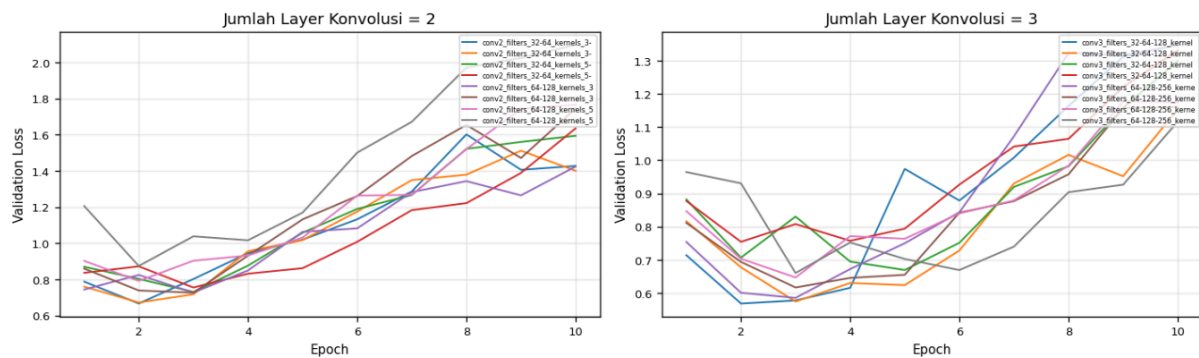
4.1.3. Pengaruh Jumlah Layer Konvolusi

Pengaruh jumlah layer konvolusi dianalisis dengan membandingkan model 2 layer dan 3 layer. Rata-rata macro F1-score untuk model dengan 2 layer konvolusi adalah 0.7391, sedangkan rata-rata macro F1-score untuk model dengan 3 layer konvolusi adalah 0.7802.

Jumlah Conv Layer	Jumlah Model	Rata-rata Macro F1	Macro F1 Terbaik
2	8	0.7391	0.7764
3	8	0.7802	0.7897



Validation Loss per Jumlah Layer Konvolusi

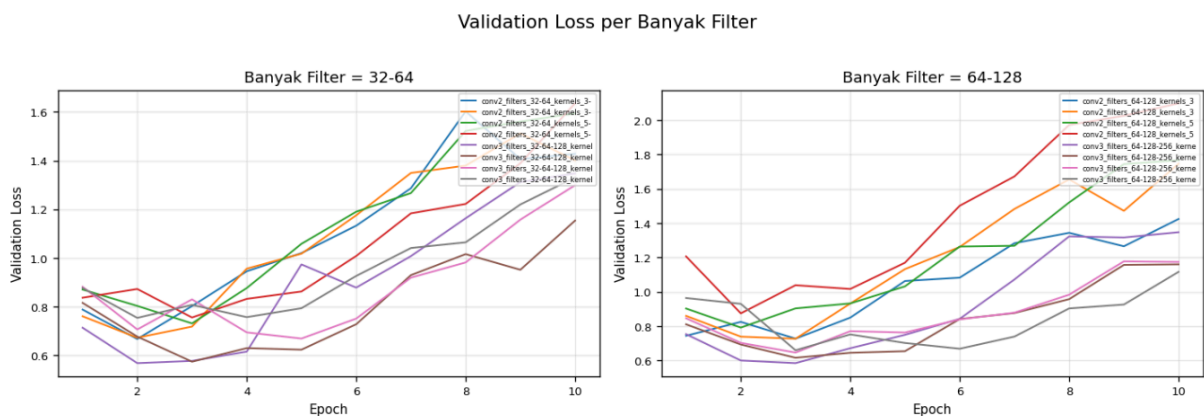
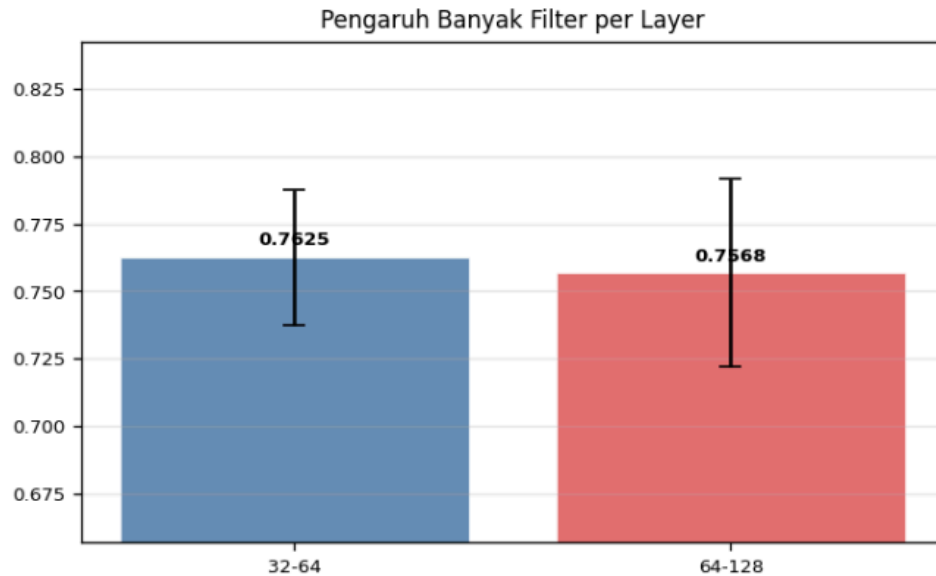


Hasil ini menunjukkan bahwa penambahan layer konvolusi meningkatkan kemampuan model dalam menangkap fitur visual yang lebih kompleks. Dengan tiga layer, model dapat membangun representasi bertingkat, mulai dari fitur sederhana seperti edge dan tekstur hingga pola objek yang lebih abstrak. Namun, kurva loss juga menunjukkan bahwa model tiga layer lebih mudah mengalami overfitting ketika dilatih terlalu lama, sehingga pemilihan epoch atau penggunaan regularisasi dapat menjadi faktor penting.

4.1.4. Pengaruh Banyak Filter

Ekspерimen filter dilakukan dengan dua skenario untuk masing-masing jumlah layer: 32-64 dan 64-128 pada model 2 layer, serta 32-64-128 dan 64-128-256 pada model 3 layer.

Filter	Jumlah Model	Rata-rata Macro F1	Macro F1 Terbaik	Rata-rata Parameter
32-64	4	0.7433	0.7663	11,252,294
64-128	4	0.7348	0.7764	22,573,318
32-64-128	4	0.7817	0.7897	5,485,254
64-128-256	4	0.7787	0.7887	11,317,766



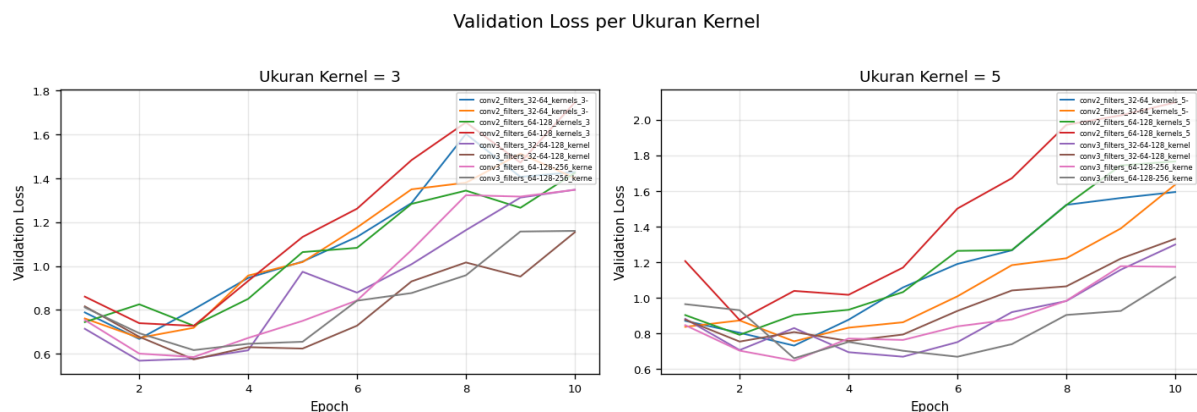
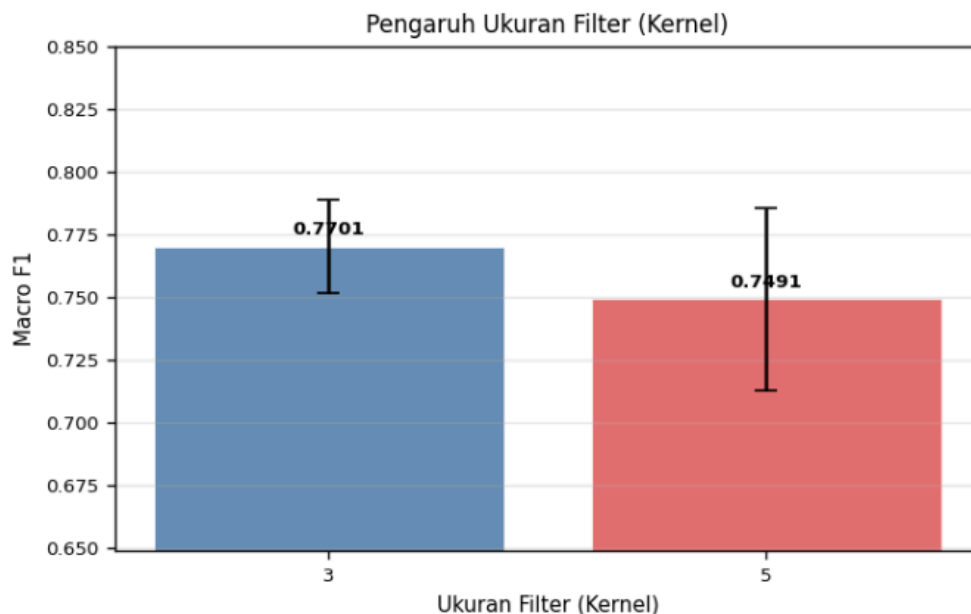
Pada model 2 layer, konfigurasi 64-128 dapat menghasilkan performa terbaik yang lebih tinggi dibanding 32-64, tetapi rata-ratanya justru lebih rendah karena kombinasi dengan average pooling dan kernel 5x5 menghasilkan performa buruk. Pada model 3 layer, filter 32-64-128 sedikit lebih baik daripada 64-128-256. Dengan kata lain, menambah jumlah filter tidak selalu meningkatkan performa. Kapasitas model yang lebih besar dapat membantu representasi, tetapi juga meningkatkan jumlah parameter dan risiko overfitting.

4.1.5. Pengaruh Ukuran Kernel

Eksperimen ukuran kernel membandingkan kernel 3x3 dan 5x5 pada jumlah layer yang sama.

Kernel	Jumlah Model	Rata-rata Macro F1	Macro F1 Terbaik
3-3	4	0.7574	0.7764
5-5	4	0.7207	0.7565

3-3-3	4	0.7828	0.7897
5-5-5	4	0.7775	0.7887

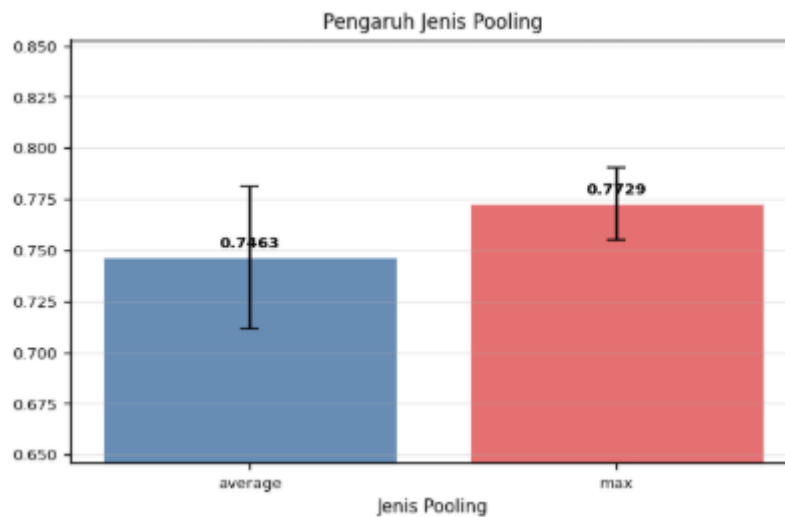


Secara umum, kernel 3x3 memberikan performa yang lebih stabil dibanding kernel 5x5. Kernel 3x3 cukup untuk menangkap pola lokal seperti edge, tekstur, dan perubahan warna, serta memiliki jumlah parameter yang lebih kecil. Kernel 5x5 memberikan receptive field yang lebih besar, tetapi pada eksperimen ini tidak memberikan peningkatan yang konsisten. Pada model 2 layer, kernel 5x5 bahkan menurunkan performa cukup besar, terutama ketika digabungkan dengan average pooling.

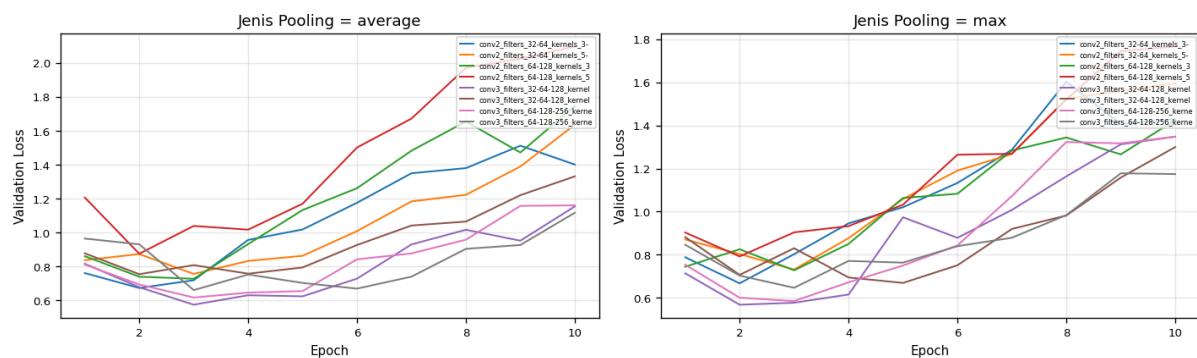
4.1.6. Pengaruh Jenis Pooling

Pooling yang dibandingkan adalah max pooling dan average pooling.

Pooling	Jumlah Model	Rata-rata Macro F1	Macro F1 Terbaik
MaxPooling2D	8	0.7729	0.7897
AveragePooling2D	8	0.7463	0.7890



Validation Loss per Jenis Pooling



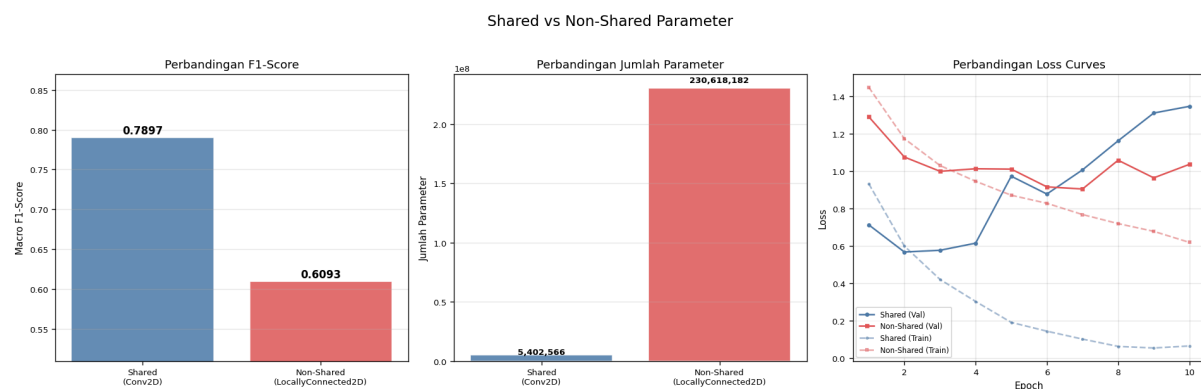
Max pooling menghasilkan rata-rata macro F1-score yang lebih tinggi dibanding average pooling. Pada dataset klasifikasi citra alam, fitur yang paling dominan seperti edge tajam, tekstur hutan, garis bangunan, dan batas antara laut atau gunung tampak penting untuk klasifikasi. Max pooling mempertahankan aktivasi terbesar pada setiap window sehingga fitur dominan tersebut tidak terlalu teredam. Sebaliknya, average pooling membuat representasi lebih halus, tetapi dapat mengurangi kekuatan fitur lokal yang penting.

4.1.7. Perbandingan Shared dan Non-Shared Parameter

Perbandingan shared dan non-shared parameter dilakukan dengan mengambil konfigurasi terbaik shared parameter, yaitu conv3_filters_32-64-128_kernels_3-3-3_pool_max, lalu

mengganti seluruh Conv2D dengan LocallyConnected2D. Model non-shared yang dihasilkan bernama local3_filters_32-64-128_kernels_3-3-3_pool_max.

Model	Jenis Parameter	Macro F1	Accuracy	Jumlah Parameter
conv3_filters_32-64-128_kernels_3-3-3_pool_max	Shared (Conv2D)	0.7897	0.7900	5,402,566
local3_filters_32-64-128_kernels_3-3-3_pool_max	Non-shared (LocallyConnected2D)	0.6093	0.6213	230,618,182



Model non-shared memiliki parameter sekitar 42,7 kali lebih banyak daripada model shared. Walaupun jumlah parameternya jauh lebih besar, performanya justru lebih rendah. Macro F1-score turun dari 0.7897 menjadi 0.6093. Hal ini menunjukkan bahwa parameter sharing pada Conv2D sangat penting untuk efisiensi dan generalisasi.

Pada Conv2D, kernel yang sama digunakan di seluruh posisi gambar. Dengan demikian, fitur seperti edge, tekstur, atau pola lokal dapat dikenali di berbagai lokasi spasial.

LocallyConnected2D tidak membagi bobot antar posisi, sehingga setiap posisi harus mempelajari polanya sendiri. Akibatnya, model memiliki kapasitas yang sangat besar, tetapi membutuhkan lebih banyak data untuk melatih setiap posisi secara efektif. Pada dataset ini, kapasitas non-shared yang terlalu besar membuat model kurang efisien dan lebih sulit melakukan generalisasi.

Kurva loss non-shared menunjukkan training loss turun dari 1.4502 menjadi 0.6201. Validation loss terbaik adalah 0.9057 pada epoch ke-7, kemudian naik lagi menjadi 1.0375 pada epoch ke-10. Dibandingkan model shared terbaik yang memiliki best validation loss

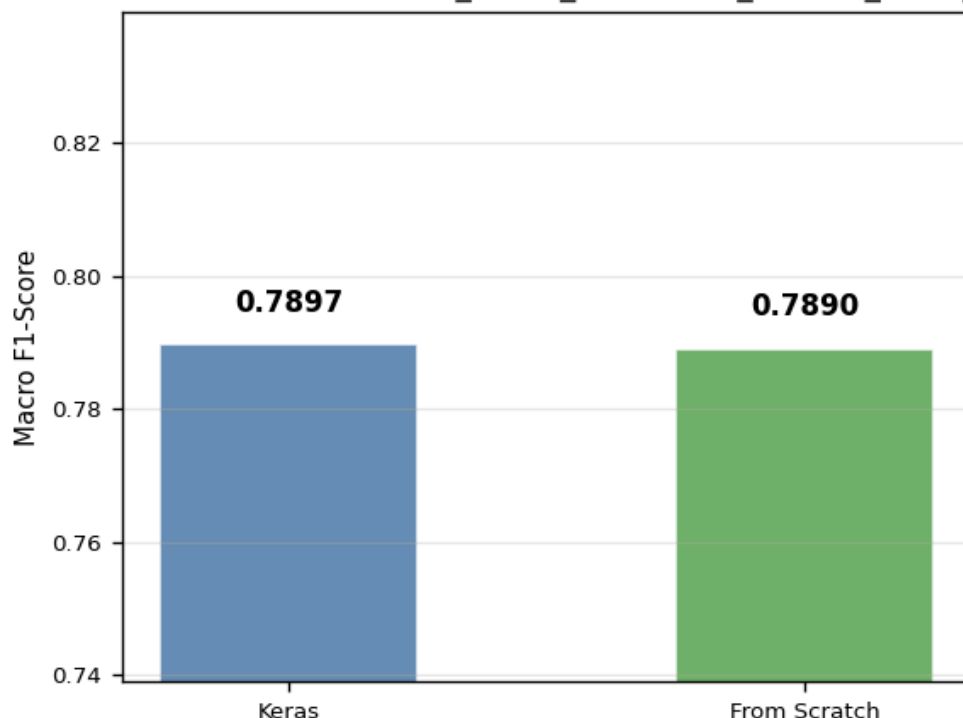
0.5686, model non-shared memiliki validation loss yang lebih tinggi. Ini mendukung kesimpulan bahwa parameter sharing membantu model belajar representasi yang lebih stabil.

4.1.8. Perbandingan Keras dan From Scratch

Forward propagation from scratch diuji dengan memuat bobot model Keras ke implementasi NumPy. Pengujian dilakukan pada model terbaik shared parameter. Prediksi model Keras dan model scratch dibandingkan menggunakan macro F1-score.

Model	Macro F1
Keras	0.789695
Scratch	0.789001
Selisih absolut	0.000694

Keras vs From Scratch — conv3_filters_32-64-128_kernels_3-3-3_pool_max



Hasil tersebut menunjukkan bahwa implementasi forward propagation from scratch sudah sangat dekat dengan Keras. Selisih macro F1-score hanya 0.000694. Perbedaan kecil ini masih wajar karena terdapat perbedaan detail operasi floating point dan cara eksekusi NumPy dibanding TensorFlow.

Selain itu, pengujian batch inference pada baseline Conv2D menghasilkan macro F1-score Keras dan scratch yang identik, yaitu 0.793490, dengan prediction agreement 1.0 pada 3000 gambar test. Hasil ini memperkuat bahwa implementasi layer Conv2D, pooling, flatten, dense, dan softmax from scratch sudah ekuivalen dengan model Keras untuk forward propagation.

Pada eksperimen scratch inference model terbaik, proses prediksi 3000 gambar dengan batch size 16 membutuhkan waktu sekitar 3 menit 1 detik. Waktu ini lebih lama dibanding Keras karena implementasi scratch menggunakan NumPy murni dan tidak memanfaatkan optimasi graf komputasi TensorFlow.

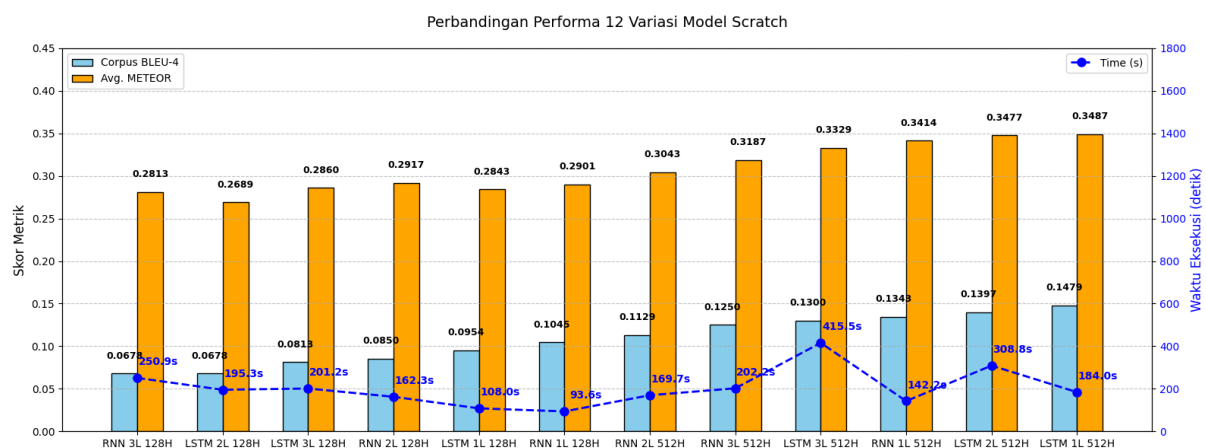
4.2. RNN dan LSTM Untuk Image Captioning

Pengujian RNN dan LSTM dikerjakan dalam konteks task image captioning menggunakan dataset Flickr8k. Kedua arsitektur (RNN dan LSTM) dilatih secara terpisah sebagai decoder dan dibandingkan hasilnya.

Proses ekstraksi fitur gambar menggunakan pretrained CNN InceptionV3 tanpa classification head (*top layer*) dan dengan bobot ImageNet yang di-*freeze*.

Dilakukan preprocessing terhadap caption dengan melakukan sanitasi, tokenisasi, dan penambahan token spesial. Proses ini menghasilkan *vocabulary* yang berisi seluruh token yang mungkin dari dataset caption, yang kemudian disimpan sebagai file .json.

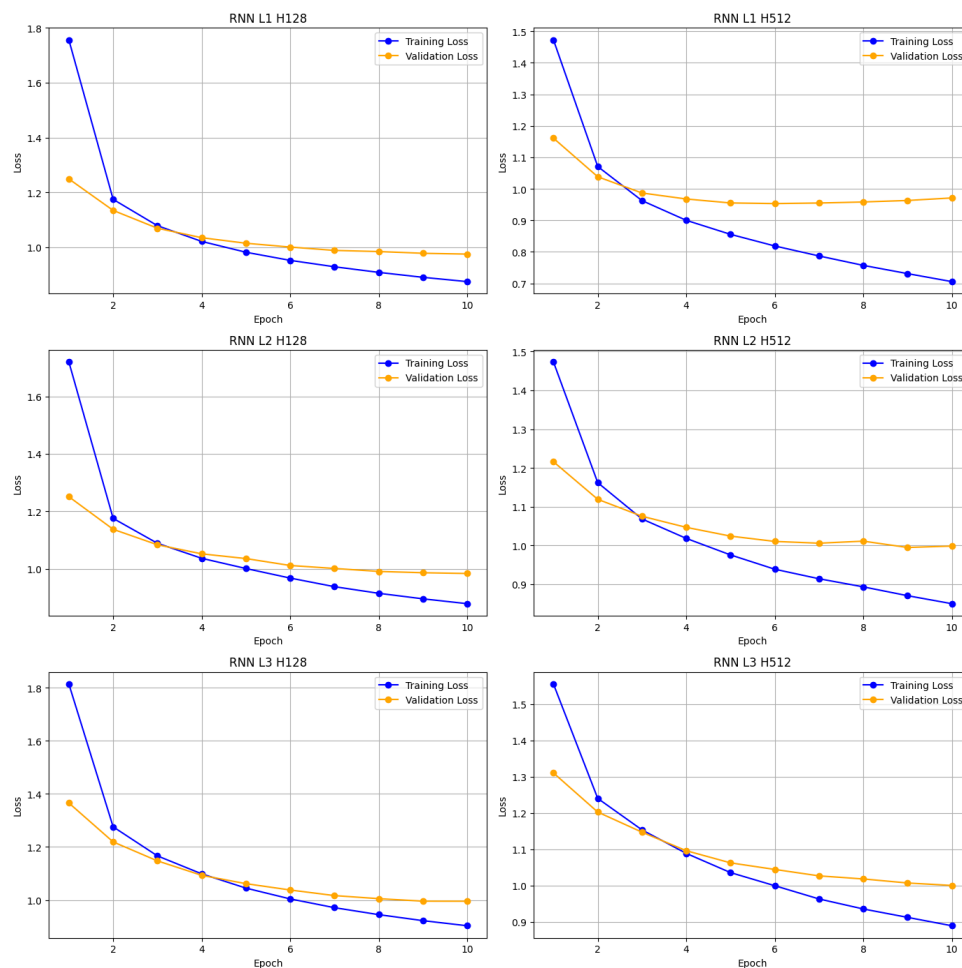
Dibawah ini adalah hasil inferensi menggunakan 12 variasi model (2 tipe, 3 jumlah layer, 2 ukuran hidden state) yang berbeda.



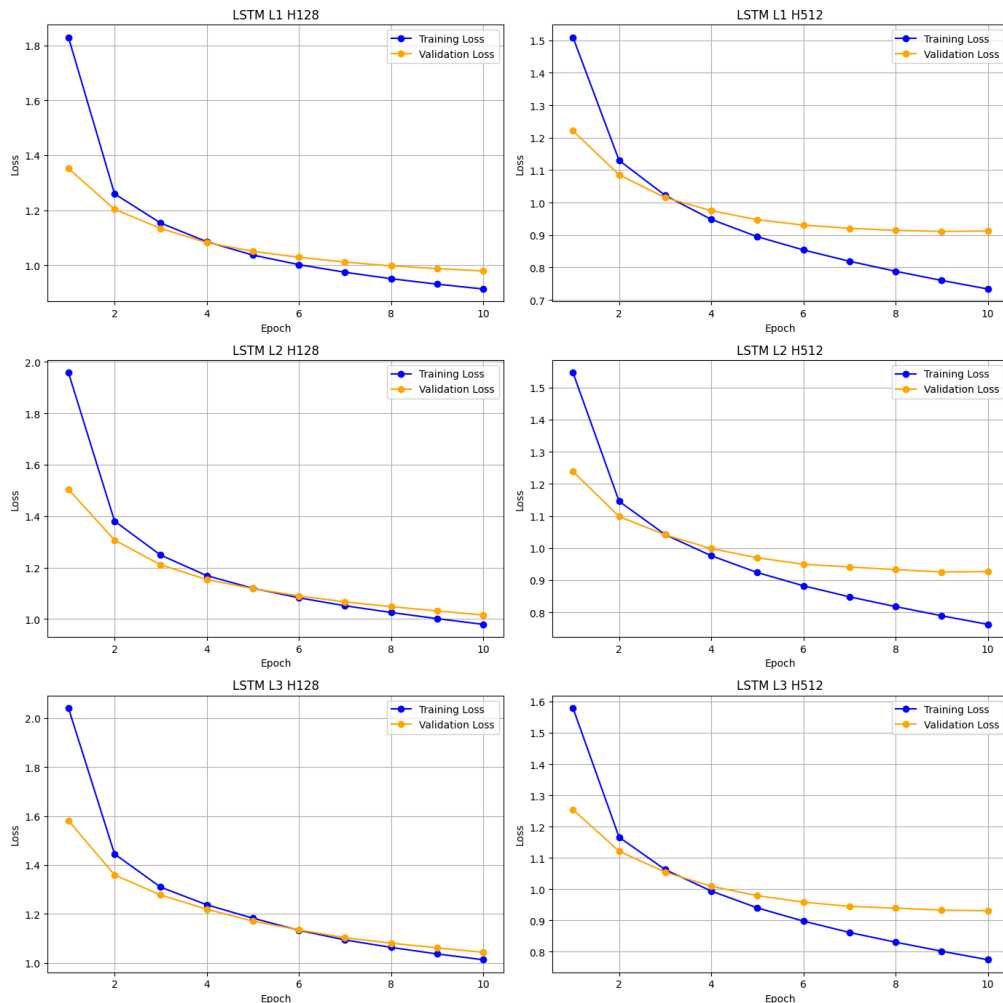
No	Model	Hidden Size	Layers	Corpus BLEU-4	Avg. METEOR	Time (s)
1	RNN	128	1	0.104465	0.290098	93.647153
2			2	0.085008	0.291654	162.253253
3			3	0.067769	0.281292	250.861453

4		512	1	0.134276	0.341362	142.224885
5			2	0.112869	0.304319	169.653029
6			3	0.123988	0.318675	202.188988
7	LSTM	128	1	0.095392	0.284296	107.990179
8			2	0.067836	0.268915	195.270376
9			3	0.081341	0.285985	201.229855
10		512	1	0.147896	0.348671	184.040311
11			2	0.139674	0.347714	308.827761
12			3	0.129956	0.332943	415.467923

Berikut adalah grafik kurva training dan validation loss untuk variasi arsitektur RNN:



Berikut adalah grafik kurva training dan validation loss untuk variasi arsitektur LSTM:



4.2.1. Perbandingan jumlah layer: RNN & LSTM

Penambahan jumlah layer tidak selalu meningkatkan performa model. Pada beberapa konfigurasi, terutama pada RNN dan LSTM dengan 3 layer, validation loss justru cenderung lebih tinggi dibandingkan model dengan 1 atau 2 layer. Kondisi ini mengindikasikan bahwa penambahan layer yang terlalu dalam dapat menyebabkan optimisasi menjadi lebih sulit, meningkatkan risiko overfitting, serta memperbesar kemungkinan terjadinya vanishing gradient, khususnya pada RNN standar. Hal tersebut terlihat pada model LSTM 3 layer hidden 128 yang memiliki validation loss sekitar 1.04, lebih buruk dibandingkan LSTM 1 layer hidden 512.

4.2.2. Perbandingan ukuran hidden state: RNN & LSTM

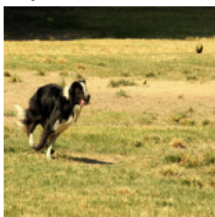
Peningkatan ukuran hidden state memberikan pengaruh yang lebih signifikan terhadap kualitas model dibandingkan peningkatan jumlah layer. Model dengan hidden size 512 secara konsisten menghasilkan nilai validation loss yang lebih rendah dibandingkan hidden size 128, baik pada arsitektur RNN maupun LSTM. Hal ini menunjukkan bahwa kapasitas representasi fitur yang lebih besar membantu model memahami pola sekuens caption dengan lebih baik sehingga menghasilkan kualitas caption yang lebih baik. Sebagai contoh, model LSTM dengan 1 layer dan hidden size 512 memperoleh validation loss sekitar 0.91, lebih baik dibandingkan konfigurasi hidden size 128 yang berada di sekitar 0.97-1.04.

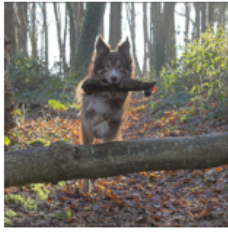
4.2.3. Perbandingan RNN vs LSTM

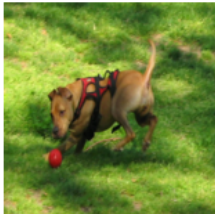
Selain mempengaruhi kualitas model, peningkatan jumlah layer dan hidden state juga berdampak langsung terhadap waktu pelatihan. Semakin besar hidden size dan semakin banyak layer yang digunakan, maka waktu komputasi meningkat secara signifikan. Pada model RNN 1 layer hidden 128, waktu pelatihan per epoch hanya sekitar 36 detik, sedangkan pada RNN 3 layer hidden 512 meningkat hingga sekitar 200 detik per epoch. Peningkatan waktu yang lebih drastis terlihat pada arsitektur LSTM karena mekanisme gate yang lebih kompleks. Model LSTM 1 layer hidden 128 memerlukan sekitar 50 detik per epoch, sementara LSTM 3 layer hidden 512 membutuhkan lebih dari 500 detik per epoch.

Secara keseluruhan, konfigurasi terbaik diperoleh pada model LSTM dengan hidden size 512 karena menghasilkan validation loss paling rendah dan kemampuan generalisasi yang lebih baik dibandingkan RNN. Namun, peningkatan performa tersebut harus dibayar dengan biaya komputasi yang jauh lebih tinggi. Oleh karena itu, pemilihan jumlah layer dan ukuran hidden state perlu mempertimbangkan trade-off antara kualitas caption yang dihasilkan dan efisiensi waktu pelatihan.

Untuk analisis kualitatif, diambil 10 sampel gambar dengan variasi skor BLEU-4 tertinggi hingga terendah untuk masing-masing arsitektur (RNN dan LSTM). Berikut adalah tabel yang berisi 10 sampel dengan skor BLEU-4 tertinggi ke terendah untuk RNN:

No.	Sampel	Tipe	BLEU-4	Caption
1	Sampel 1 (Index 685) 	Ground Truth	-	a brown dog and a gray dog are playing in the snow
		RNN	1.0000	two dogs play in the snow
		LSTM	0.0974	a dog is running through the snow
2	Sampel 2 (Index 1532) 	Ground Truth	-	a black and white dog is chasing a ball on an open plane
		RNN	0.9622	a black and white dog is running through a field
		LSTM	0.9147	a black and white dog is running through a grassy field

3	Sampel 3 (Index 618) 	Ground Truth	-	a dog is jumping over a log in a wooded area while carrying another log
		RNN	0.9457	a dog is jumping over a fallen tree
		LSTM	0.8669	a dog is jumping over a log
4	Sampel 4 (Index 1332) 	Ground Truth	-	a group of little girls perform a recital onstage dressed in all white
		RNN	0.0891	a large group of people are sitting on a fire
		LSTM	0.0385	a crowd of people are in a <unk> race
5	Sampel 5 (Index 584) 	Ground Truth	-	there are two kids in swimmies playing outside next to some trees
		RNN	0.0891	a young girl wearing a pink shirt is standing on a rock in the middle of the water
		LSTM	0.0847	a young girl is climbing a rock wall
6	Sampel 6 (Index 369) 	Ground Truth	-	a homeless man being observed by a man in business attire
		RNN	0.0889	a man and woman walk down the street
		LSTM	0.1106	a man in a black jacket is standing in front of a <unk>
7	Sampel 7 (Index 762) 	Ground Truth	-	a large and small dog interact in a grassy field
		RNN	0.0889	two dogs are playing together in the grass
		LSTM	0.0748	a brown dog is running through the grass

8	Sampel 8 (Index 298) 	Ground Truth	-	a brown and white corgi trailing a leash chasing sheep
		RNN	0.0000	three dogs are playing together in the grass
		LSTM	0.0847	a dog is running through a grassy field
9	Sampel 9 (Index 1105) 	Ground Truth	-	a brown dog in a red harness chasing a red ball
		RNN	0.0000	two dogs are running through the grass
		LSTM	0.1774	a brown dog is running through the grass
10	Sampel 10 (Index 1485) 	Ground Truth	-	two guys are playing horse shoe together
		RNN	0.0000	a man in a blue shirt is running on the grass
		LSTM	0.0000	a man in a blue shirt is throwing a ball into a lake

Berdasarkan analisis kualitatif hasil kualitas caption, terlihat pola perbedaan yang signifikan dalam kemampuan generatif antara arsitektur RNN dan LSTM pada tugas image captioning. Pada kategori skor tinggi, RNN mampu menghasilkan kalimat yang sangat akurat untuk skenario sederhana dengan subjek yang jelas, seperti deskripsi dua anjing yang bermain di salju. Namun, saat dihadapkan pada skenario yang lebih kompleks di kategori medium, RNN mulai menunjukkan pola "halusinasi" atau ketidakakuratan detail, seperti mendeskripsikan seorang gadis mengenakan baju merah muda yang sedang berdiri di atas batu, padahal teks referensi asli menyebutkan dua anak kecil yang sedang bermain. Hal ini mengindikasikan bahwa RNN cenderung kesulitan mempertahankan informasi fitur gambar yang diterima di awal sekuensi seiring bertambahnya panjang kalimat yang dihasilkan.

Berikut adalah tabel yang berisi 10 sampel dengan skor BLEU-4 tertinggi ke terendah untuk LSTM:

No.	Sampel	Tipe	BLEU-4	Caption
-----	--------	------	--------	---------

1	Sampel 1 (Index 106) 	Ground Truth	-	a man dressed in black is surfing on a large blue wave
		RNN	0.4047	a surfer rides a wave
		LSTM	1.0000	a surfer is riding a wave
2	Sampel 2 (Index 273) 	Ground Truth	-	a black dog carries a white toy in his mouth and walks on the snowy field
		RNN	0.5946	a black dog running through the snow
		LSTM	1.0000	a black dog is running through the snow
3	Sampel 3 (Index 504) 	Ground Truth	-	a black dog jumping into some water to catch a red and blue frisbee
		RNN	0.1159	a <unk> <unk> <unk> in the water
		LSTM	1.0000	a dog is jumping into the water
4	Sampel 4 (Index 1181) 	Ground Truth	-	a little blonde girl splashing in a mud puddle
		RNN	0.0604	a young boy wearing a blue shirt is playing in the water
		LSTM	0.0940	a young girl is playing in the water
5	Sampel 5 (Index 960) 	Ground Truth	-	a dirt biker rides up a rocky hill on a motorized dirt bike
		RNN	0.1891	a man is riding a bicycle on a dirt path
		LSTM	0.0937	a man rides a bicycle down a hill

6	Sampel 6 (Index 1297) 	Ground Truth	-	a child is jumping into a swimming pool
		RNN	0.0713	a young boy in a blue shirt is jumping in the air
		LSTM	0.0937	a young boy is jumping on a trampoline
7	Sampel 7 (Index 1390) 	Ground Truth	-	two dogs play in the grass
		RNN	0.4111	two white dogs are running through the grass
		LSTM	0.0937	two dogs are running through a grassy field
8	Sampel 8 (Index 329) 	Ground Truth	-	the two dogs are interested in the gravel ground surrounded by concrete
		RNN	0.0340	a small dog is running through the grass
		LSTM	0.0000	a dog is jumping up a hurdle
9	Sampel 9 (Index 1038) 	Ground Truth	-	the three dogs are playing with each other
		RNN	0.1774	two dogs are playing in a grassy area
		LSTM	0.0000	a black dog is biting a smaller dog
10	Sampel 10 (Index 1485) 	Ground Truth	-	two guys are playing horse shoe together
		RNN	0.0000	a man in a blue shirt is running on the grass
		LSTM	0.0000	a man in a blue shirt is throwing a ball into a lake

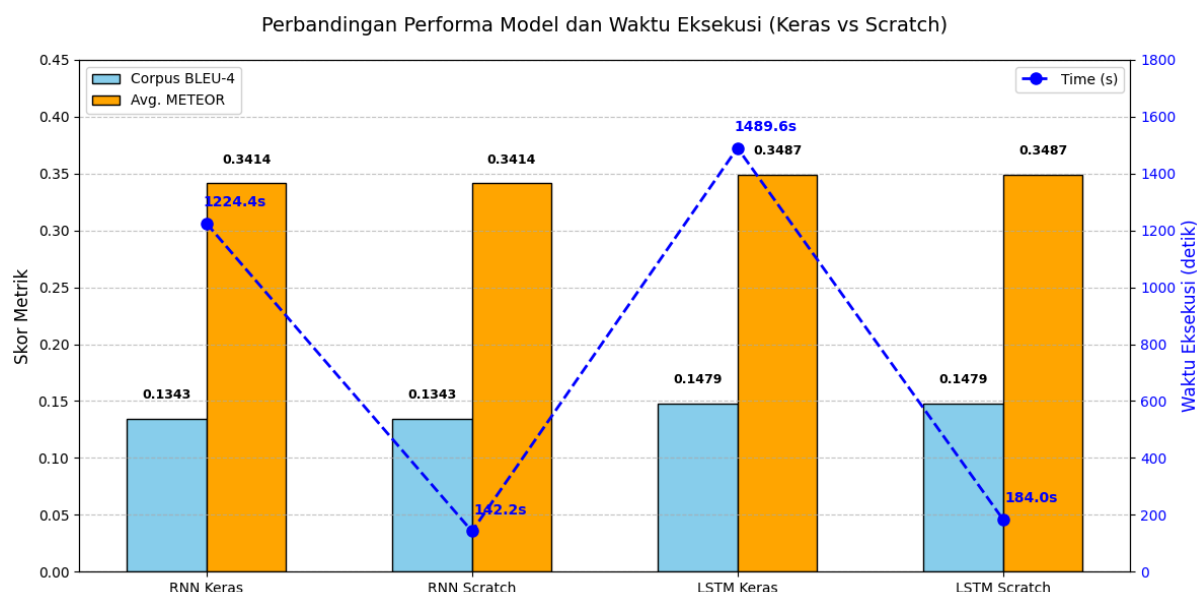
Sebaliknya, arsitektur LSTM menunjukkan konsistensi yang lebih baik dalam menangkap aksi dan hubungan antar objek dalam gambar. Pada kategori skor tinggi, LSTM berhasil

mencapai skor BLEU sempurna (1.0000) untuk deskripsi aktivitas seperti peselancar yang menaiki ombak atau anjing hitam yang berlari di salju. Keunggulan utama LSTM terlihat pada kemampuannya menyusun struktur kalimat subjek-predikat yang lebih koheren pada tingkat kesulitan menengah, misalnya dalam mendeskripsikan pengendara motor di perbukitan berbatu atau anak-anak yang bermain air, meskipun detail spesifiknya terkadang masih meleset dari teks referensi. Pada kategori skor rendah, kedua model seringkali gagal menangkap konteks aktivitas yang sangat spesifik atau jarang ditemui dalam data latih, seperti permainan *horse shoe* yang justru diprediksi sebagai orang yang berlari atau melempar bola ke danau.

Secara teoritis, perbedaan kualitas caption ini berkaitan erat dengan masalah vanishing gradient dan keterbatasan memori jangka panjang pada RNN. Dalam arsitektur RNN, gradien yang dikirim balik saat proses pelatihan cenderung mengecil secara eksponensial seiring bertambahnya time step. Akibatnya, informasi penting dari fitur gambar yang dimasukkan pada langkah awal $t - 1$ seringkali terlupakan oleh model saat harus memprediksi kata-kata di akhir kalimat. Hal ini menjelaskan mengapa RNN sering menghasilkan deskripsi yang kurang detail atau kehilangan konteks subjek di tengah kalimat.

Di sisi lain, LSTM dirancang khusus untuk mengatasi masalah tersebut melalui mekanisme gate (input, forget, dan output) serta komponen cell state. Cell state berfungsi sebagai jalur informasi jangka panjang yang memungkinkan model untuk secara selektif menyimpan atau menghapus informasi penting dari fitur gambar di sepanjang proses pembentukan kalimat. Kemampuan memori jangka panjang ini memungkinkan LSTM untuk mempertahankan ketergantungan temporal yang lebih kuat, sehingga caption yang dihasilkan tetap relevan dengan konteks gambar asli dari awal hingga akhir sekuensi. Oleh karena itu, LSTM umumnya menghasilkan performa yang lebih unggul dibandingkan RNN, terutama pada kalimat yang membutuhkan pemahaman konteks yang lebih mendalam.

4.2.4. Perbandingan Keras vs from scratch



No.	Model	Implementasi	Corpus BLEU-4	Avg. METEOR	Time (s)
1	RNN	Keras	0.134276	0.341362	1224.417219
2		Scratch	0.134276	0.341362	142.224885
3	LSTM	Keras	0.147896	0.348761	1489.628119
4		Scratch	0.147896	0.348761	184.040311

Berdasarkan hasil eksperimen, terdapat pola yang sangat jelas mengenai perbandingan antara implementasi menggunakan Keras dengan implementasi scratch menggunakan NumPy. Dari sisi kualitas metrik, skor BLEU-4 dan METEOR yang dihasilkan oleh kedua implementasi untuk model yang sama menunjukkan nilai yang identik hingga beberapa angka di belakang koma. Hal ini membuktikan bahwa logika forward propagation yang dibuat dari scratch telah berhasil mereplikasi struktur komputasi Keras dengan sangat akurat, termasuk pemindahan bobot (weight loading) dari model .h5 ke dalam matriks NumPy. Selain itu, secara konsisten model LSTM mengungguli model RNN pada kedua implementasi, dengan skor BLEU-4 sebesar 0,1479 dibandingkan 0,1343, yang menegaskan superioritas LSTM dalam menangani ketergantungan sekuensial jangka panjang.

Perbedaan terlihat pada waktu eksekusi proses inferensi. Implementasi from scratch berjalan jauh lebih cepat dibandingkan dengan Keras, dengan rasio efisiensi mencapai lebih dari ~8 kali lipat. Sebagai contoh, model LSTM from scratch hanya membutuhkan waktu sekitar 184 detik, sementara versi Keras memerlukan waktu hingga 1489 detik untuk menyelesaikan proses yang sama. Hal ini didasari oleh asumsi adanya overhead pada Keras saat menangani operasi inference secara atomik (satu per satu per kata) dalam mode greedy decoding. Saat melakukan prediksi kata demi kata, Keras melakukan inisialisasi internal, pengecekan graf, dan manajemen memori untuk setiap pemanggilan fungsi predict(). Sebaliknya, implementasi NumPy melakukan operasi matriks langsung pada CPU tanpa lapisan abstraksi tambahan, sehingga jauh lebih efisien untuk proses inferensi sekuensial yang tidak memanfaatkan paralelisasi batch besar.

4.2.5. Pengaruh panjang maksimum caption terhadap BLEU-4

No.	Max Length	Corpus BLEU-4	Avg. METEOR	Time (s)
1	10	0.159629	0.340115	155.517466
2	20	0.148625	0.348520	159.041371
3	30	0.147896	0.348671	157.594217

Berdasarkan hasil pengujian tersebut, variasi panjang caption memiliki pengaruh yang signifikan terhadap skor BLEU-4 dan kualitas linguistik dari caption yang dihasilkan. Pada pengaturan panjang maksimum yang sangat pendek (Max Length 10), model secara konsisten

menghasilkan kalimat yang terpotong dan tidak lengkap. Hal ini menyebabkan skor BLEU-4 cenderung rendah karena informasi krusial mengenai aksi atau objek latar belakang gagal ditangkap secara utuh. Namun, pada beberapa kasus dengan subjek tunggal yang sangat kuat, skor pada panjang 10 bisa terlihat cukup tinggi secara artifisial karena model langsung mengenai kata kunci utama sebelum kalimat terpotong.

Peningkatan panjang maksimum ke variasi menengah (Max Length 20) umumnya memberikan dampak positif pada kelengkapan kalimat, yang dalam beberapa sampel meningkatkan skor BLEU-4 dibandingkan panjang 10. Pada tahap ini, model mampu menyusun deskripsi yang lebih detail mengenai pakaian atau interaksi antar objek. Meskipun demikian, terdapat kecenderungan di mana peningkatan panjang dari 20 ke 40 justru menurunkan skor BLEU-4. Penurunan ini terjadi karena model seringkali mulai melakukan pengulangan kata atau menambahkan detail yang tidak relevan (halusinasi) demi memenuhi kuota panjang sekuensi seperti pengulangan frasa.

Kesimpulannya, terdapat ambang batas optimal untuk panjang caption di mana penambahan panjang sekuensi tidak selalu berbanding lurus dengan peningkatan kualitas skor. Batasan panjang yang terlalu pendek menyebabkan hilangnya informasi penting (under-generation), sementara batasan yang terlalu panjang tanpa mekanisme *early termination* yang tepat memicu model untuk menghasilkan kata-kata repetitif yang menurunkan skor BLEU-4. Fenomena ini menunjukkan bahwa model LSTM memiliki kecenderungan untuk terus menghasilkan kata berdasarkan probabilitas distribusi meskipun deskripsi visual inti telah terpenuhi, yang pada akhirnya justru memperlebar jarak antara kalimat prediksi dengan caption sebenarnya.

BAB V

KESIMPULAN & SARAN

5.1. Kesimpulan

1. Implementasi forward propagation CNN, RNN, dan LSTM from scratch berhasil dibuat secara modular menggunakan NumPy. Model scratch tidak melakukan training ulang, melainkan memuat bobot dari model Keras sehingga hasil forward propagation dapat dibandingkan langsung dengan model pembandingan. Hasil pengujian menunjukkan bahwa implementasi scratch sudah ekuivalen dengan Keras, baik pada klasifikasi gambar maupun image captioning.
2. Pada task CNN image classification, model terbaik dari 16 konfigurasi Conv2D shared parameter adalah konfigurasi tiga layer konvolusi dengan filter 32-64-128, kernel 3x3 pada setiap layer, dan max pooling. Model ini memperoleh macro F1-score sebesar 0.7897 dan accuracy sebesar 0.7900. Secara umum, penambahan layer konvolusi dari dua menjadi tiga layer meningkatkan rata-rata macro F1-score, sedangkan kernel 3x3 dan max pooling memberikan hasil yang lebih stabil dibandingkan kernel 5x5 dan average pooling.
3. Perbandingan shared dan non-shared parameter menunjukkan bahwa parameter sharing pada Conv2D sangat penting untuk efisiensi dan generalisasi. Model shared terbaik memiliki sekitar 5.4 juta parameter dan macro F1-score 0.7897, sedangkan model non-shared LocallyConnected2D memiliki sekitar 230.6 juta parameter tetapi macro F1-score turun menjadi 0.6093. Hal ini menunjukkan bahwa jumlah parameter yang lebih besar tidak selalu menghasilkan performa lebih baik jika tidak diimbangi dengan generalisasi yang baik.
4. Perbandingan Keras dan from scratch pada CNN menunjukkan selisih macro F1-score yang sangat kecil, yaitu sekitar 0.000694 pada model terbaik shared parameter. Selain itu, batch inference baseline Conv2D menghasilkan prediction agreement 1.0 pada 3000 gambar test. Hasil ini memperkuat bahwa implementasi Conv2D, pooling, flatten, dense, softmax, dan mekanisme pemuatan bobot dari Keras sudah berjalan dengan benar.
5. Pada task image captioning, peningkatan ukuran hidden state memberikan pengaruh yang lebih signifikan dibandingkan penambahan jumlah layer. Model dengan hidden size 512 cenderung memiliki kualitas caption yang lebih baik dibandingkan hidden size 128. Namun, penambahan jumlah layer tidak selalu meningkatkan performa karena dapat membuat optimisasi lebih sulit dan meningkatkan waktu komputasi.
6. Secara keseluruhan, LSTM menghasilkan performa captioning yang lebih baik dibandingkan RNN. Model LSTM terbaik memperoleh skor BLEU-4 sebesar 0.147896 dan skor METEOR sebesar 0.348761, lebih tinggi dibandingkan RNN terbaik dengan skor BLEU-4 sebesar 0.134276 dan skor METEOR sebesar 0.341362. Keunggulan LSTM terutama disebabkan oleh adanya gate dan cell state yang

membantu mempertahankan informasi jangka panjang dari fitur gambar dan token sebelumnya.

7. Variasi panjang maksimum caption mempengaruhi keseimbangan antara kelengkapan kalimat dan risiko pengulangan. Batas panjang yang terlalu pendek dapat menyebabkan caption terpotong, sedangkan batas panjang yang terlalu panjang dapat memicu repetisi atau halusinasi. Oleh karena itu, pemilihan panjang maksimum caption perlu mempertimbangkan kualitas metrik dan kualitas linguistik caption secara kualitatif.

5.2. Saran

1. Untuk eksperimen CNN, disarankan menambahkan regularisasi seperti dropout, L2 regularization, dan early stopping agar overfitting dapat dikurangi. Hal ini penting karena beberapa kurva validation loss meningkat setelah beberapa epoch meskipun training loss terus menurun.
2. Untuk model non-shared parameter, diperlukan strategi pengurangan kompleksitas agar mudah digunakan kembali. Jumlah parameter LocallyConnected2D yang sangat besar membuat model sulit dilatih dan kurang efisien. Pengujian lanjutan dapat mencoba ukuran input lebih kecil, jumlah filter lebih sedikit, atau regularisasi yang lebih kuat.
3. Strategi decoding dapat dikembangkan dari greedy decoding menjadi beam search. Greedy decoding hanya memilih kata dengan probabilitas terbesar pada setiap timestep, sehingga mudah terjebak pada pilihan lokal. Beam search dapat mengevaluasi beberapa kandidat kalimat sekaligus dan berpotensi menghasilkan caption lebih baik.
4. Evaluasi image captioning sebaiknya tidak hanya mengandalkan BLEU-4 dan METEOR, tetapi juga dapat dilengkapi dengan metrik lain seperti CIDEr atau ROUGE-L.

REFERENSI

- [Flickr8k dataset \(Kaggle\)](#)
- [Intel Image Classification dataset \(Kaggle\)](#)
- [d2l.ai: LSTM](#)
- [d2l.ai: Image Captioning](#)
- <https://medium.com/latinxinai/convolutional-neural-network-from-scratch-6b1c856e1c07>
- <https://medium.com/@thisislong/building-a-recurrent-neural-network-from-scratch-ba9b27a42856>
- <https://medium.com/@CallMeTwitch/building-a-neural-network-zoo-from-scratch-the-long-short-term-memory-network-1cec5cf31b7>

LAMPIRAN

Link Repositori

<https://github.com/Azzkaaaa/Tugas-Besar-ML-2>

Tabel Pembagian Tugas

NIM	BAGIAN
13523135	• Eksperiment CNN
13523137	• Fungsi Aktivasi • Image loader • CNN from scratch • Locally Connected from scratch • Laporan • Batch Inference CNN
13523162	• RNN • LSTM • Eksperiment RNN • Laporan

FORMAT DEKLARASI PENGGUNAAN AI SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:

Nama/NIM : Ahmad Syafiq/13523135, Muhammad Aulia Azka/13523137, Fachriza Ahmad Setyono/13523162

Fakultas/Sekolah : STEI/Teknik Informatika

Kode Mata Kuliah : IF3270

Nama Mata Kuliah : Pembelajaran Mesin

Judul Tugas/Proposal : Pembelajaran Mesin Convolutional Neural Network dan Recurrent Neural Network

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	-

Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Tidak	-
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	3
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	-
Lainnya, Jelaskan:		-

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 18 Maret 2026



Ahmad Syafiq
13523135